

Chroniques du Kraken oublié

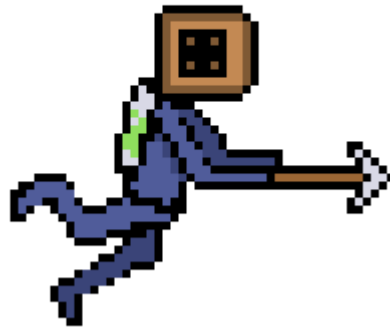


Figure 1: Image du jeu

Imad El Khattabi
Chef de projet : Pascal Benzonana
Expert no°1: Flavio Pacifico
Expert no°2: Hélder Costa Lopes

Table des matières

1	Analyse préliminaire	3
1.1	Introduction	3
1.2	Objectifs.....	3
1.3	Planification initiale	5
2	Analyse / Conception	6
2.1	Concept	6
2.2	Maquettes :.....	8
2.3	Diagramme de Classe	12
2.4	MCD et MLD	14
2.5	Choix du Code	15
2.6	Stratégie de test.....	15
2.7	Risques techniques	18
2.8	Planification	18
2.9	Dossier de conception	18
3	Réalisation	19
3.1	Dossier de réalisation	19
3.2	Description des tests effectués plus résultats des tests.....	21
3.3	Codes Particuliers.....	57
3.4	Erreurs restantes	62
3.5	Améliorations possibles	63
3.6	Liste des documents fournis	63
4	Conclusions	63
5	Annexes.....	64
5.1	Planification Initial illustré.....	64
5.2	Glossaire	65
5.3	Table des illustrations	67
5.4	Résumé du rapport du TPI	69
5.5	Sources – Bibliographie.....	69
5.6	Journal de travail	70
5.7	Manuel d'Installation	72
5.8	Manuel d'Utilisation.....	72

1 Analyse préliminaire

1.1 Introduction

Le monde du jeu vidéo est un monde très vaste qui regroupe plusieurs genres et plusieurs types de joueurs. Plusieurs personnes passent leur temps à jouer sans se soucier du temps et de la quantité de travail que cela a dû demander pour les développeurs et les designers de concevoir le jeu, quel que soit le genre.

C'est dans cette optique que j'ai décidé de réaliser mon TPI afin de me mettre à la place des développeurs ainsi que des designers afin d'évaluer la quantité de travail nécessaire et ainsi déterminer si la pression que plusieurs grandes entreprises du jeu vidéo imposent à leurs développeurs est justifiée ou non.

Ce projet me permettra donc à créer un jeu vidéo en 2D intitulé *Les Chroniques du Kraken oublié*, dans lequel nous incarnerons un plongeur à la recherche de la cité mythique *AQUAELIS* afin de mettre la main sur son légendaire trésor en explorant un monde marin rempli d'ennemis et d'énigmes. Un projet qui me permettra de voir à quel point créer un jeu peut être ambitieux même si celui-ci est petit.

1.2 Objectifs

Les objectifs de ce projet seront :

- Avoir un système de connexion
 - Avoir un écran de création de compte (pseudo, mot de passe)
 - Avoir un écran de connexion (vérification de BDD)
 - Avoir un historique de de scores pour chaque joueur
 - Avoir une base de données locale (MySQL)
- Avoir un système de score
 - Pouvoir afficher le score en temps réel
 - Gagner des points en :
 - Battant des ennemis
 - Collectant des trésors
 - Résolvant des énigmes
 - Gagner des points bonus avec le temps restant
 - Avoir des pénalités de score en :
 - Prenant des pièges ou en perdant
 - Sauvegarder le score de chaque jouer dans la BDD
- Avoir un système de sauvegarde
 - Avoir une sauvegarde automatique
 - Avoir une sauvegarde manuelle
 - La sauvegarde doit :
 - Avoir l'ID du joueur
 - La zone actuelle

- Le nombre de PV actuel
 - Avoir les mêmes items dans l'inventaire
 - Avoir le bon score
 - Avoir une progression des énigmes
- Pouvoir continuer la partie en cours depuis le menu d'accueil
- Avoir un système de déplacement avec les touches fléchées
 - Avoir une animation de mouvement pour le joueur lors du déplacement
- Avoir un système de collision
 - Ne pas pouvoir traverser les murs, objets, et les bords de l'écran
 - Ne pas pouvoir traverser les ennemis avec le système de « hitbox »
 - Gérer les interactions avec les portes qui s'ouvrent
- Avoir un système de combat simple
 - Pouvoir attaquer au contact
 - Avoir une animation lorsque le joueur attaque
 - Pouvoir recevoir des dégâts lorsqu'un ennemi nous touche
 - Dégâts subis variables selon l'ennemi
- Avoir un système de point de vie
 - Afficher la barre de points de vie
 - Pouvoir prendre des dégâts diminuant ainsi les points de vie
 - Mourir lorsque les points de vie tombent à zéro
 - Pouvoir se soigner
- Avoir un système d'inventaire
 - Pouvoir ramasser des items et les garder avec soi
 - Pouvoir sélectionner un item spécifique
 - Pouvoir utiliser les items stockés dans l'inventaire
 - Avoir une limite d'items stockés dans l'inventaire (max. 8 items)
- Avoir un système de changement de map
 - Pouvoir se diriger vers un bord d'une map actuel et atterrir vers la bonne map
- Avoir un menu principal contenant
 - Le titre du jeu
 - Un bouton New Game
 - Un bouton Continue
 - Un bouton Classement
 - Un bouton Quitter
- Avoir un menu pause contenant
 - Un bouton Reprendre
 - Un bouton Contrôles
 - Un bouton Sauvegarder
 - Un bouton Quitter

1.3 Planification initiale

J'ai structuré ma planification en utilisant la méthode agile afin d'être plus flexible dans mes stories si jamais je devais reporter certaines ou non. Elle est découpée en deux sprints. Le premier commence le 23.04.2026 et se termine le 07.05.2026. Il contient le plus de stories qui sont en réalité toute la conception du jeu, c'est-à-dire le design, le gameplay le changement de map, le système de collision de combat et l'inventaire.

Le deuxième sprint débutera le 11.05.2026 et se terminera le 19.05.2026. Il sera destiné à gestion du score, à la création de la base de données afin de gérer les inscriptions des joueurs ainsi que leur connexion et la sauvegarde du Score, et enfin la création du système de Sauvegarde ainsi que le système de Menu.

Afin d'avoir un aperçu plus visuel de la planification, celle-ci est présentée en annexe à la fin du rapport.

2 Analyse / Conception

2.1 Concept

Pour ce projet, plusieurs fonctionnalités ont été discutées. Parmi celles qui ont été envisagées voici celles qui ont été retenues :

1. Système d'inscription/connexion :
Le jeu possèdera un système d'inscription et de connexion afin de pouvoir identifier les joueurs et différencier leur partie des autres joueurs.
2. Mouvement du joueur :
Le joueur pourra se déplacer sur l'écran en utilisant les touches fléchées du clavier, il ne pourra pas traverser les bords de l'écran.
3. Système de caméra :
Le jeu contiendra un système de caméra qui n'affichera pas toute la map mais une partie afin que le joueur puisse explorer la map sans savoir ce qui l'attend.
4. Point de vie du joueur :
Le joueur possèdera une barre de vie qui s'affichera en haut à gauche de l'écran. Elle sera limitée à 100 hp comme dans la plupart des jeux et la partie se termine lorsque le joueur n'a plus de point de vie.
5. Système d'inventaire et d'interaction avec l'environnement pour le joueur :
Le jeu aura un système d'inventaire et d'interaction avec l'environnement pour que le joueur puisse acquérir différents objets et/ou débloquent de nouvelles portes.
6. Système d'attaque pour le joueur :
Le jeu comportera un système d'attaque pour que le joueur puisse se défendre lorsque des ennemis s'approchent de lui pour l'attaquer.
7. Système de soin :
Le jeu comportera un système de soin pour que le joueur puisse se régénérer s'il a pris des dégâts. Il sera principalement possible grâce à des potions de soin à trouver.
8. Système de collision :
Le jeu possèdera un système de collision qui empêchera le joueur de traverser les ennemis et les objets, limitant ainsi ses déplacements. Il en sera de même pour les ennemis, qui ne pourront pas non plus, traverser le joueur ou les objets.

9. Changement de map pour se déplacer dans le monde :
Le monde sera illustré sous forme de tableau et le joueur pourra s'y déplacer en s'approchant des bords du tableau afin de donner une sensation de grandeur au monde.
10. Système de temps :
Le jeu possèdera un système de compte à rebours qui sera surtout utilisé pour comptabiliser le nombre de points bonus que le joueur recevra s'il termine le jeu.
11. Un système de Score :
Le jeu contiendra également un système de Score qui augmentera lorsqu'il vaincra des ennemis, récoltera des perles en guise de trésor ou lorsqu'il résolvera des énigmes.

Cependant, si le joueur se prend des dégâts ou s'il meurt, il verra son Score diminuer, ce qui rajoute de la difficulté à obtenir un bon Score.

Et enfin lorsque le joueur termine le jeu il bénéficiera de points Bonus si son temps ne s'est pas écoulé.

Ce Score sera conservé dans une base de données ainsi que son ID afin d'avoir un classement à la fin des joueurs ayant le meilleur Score.
12. Un Système de sauvegarde :
Le jeu possèdera un système de sauvegarde qui sauvegardera l'état du monde, la position du joueur ses points de vie son inventaire, son Score, son temps restant, son pourcentage d'énigme résolu ainsi que son ID.

Ce fichier sera également lié à la base de données afin de ne pas mélanger les sauvegardes d'autres joueur.

2.2 Maquettes :

Page d'inscription :



Titre du Jeu

Username:

Password:

Confirme password:

Sign in

already have an account / Log in

Figure 2: Maquette de la page d'inscription

Page de connexion :



Titre du Jeu

Username:

Password:

Login

don't have an account / sign in

Figure 3: Maquette de la page de connexion

Page du menu d'accueil sans sauvegarde :



Figure 4: Maquette du jeu sans sauvegarde créée

Page du menu d'accueil avec sauvegarde créée :

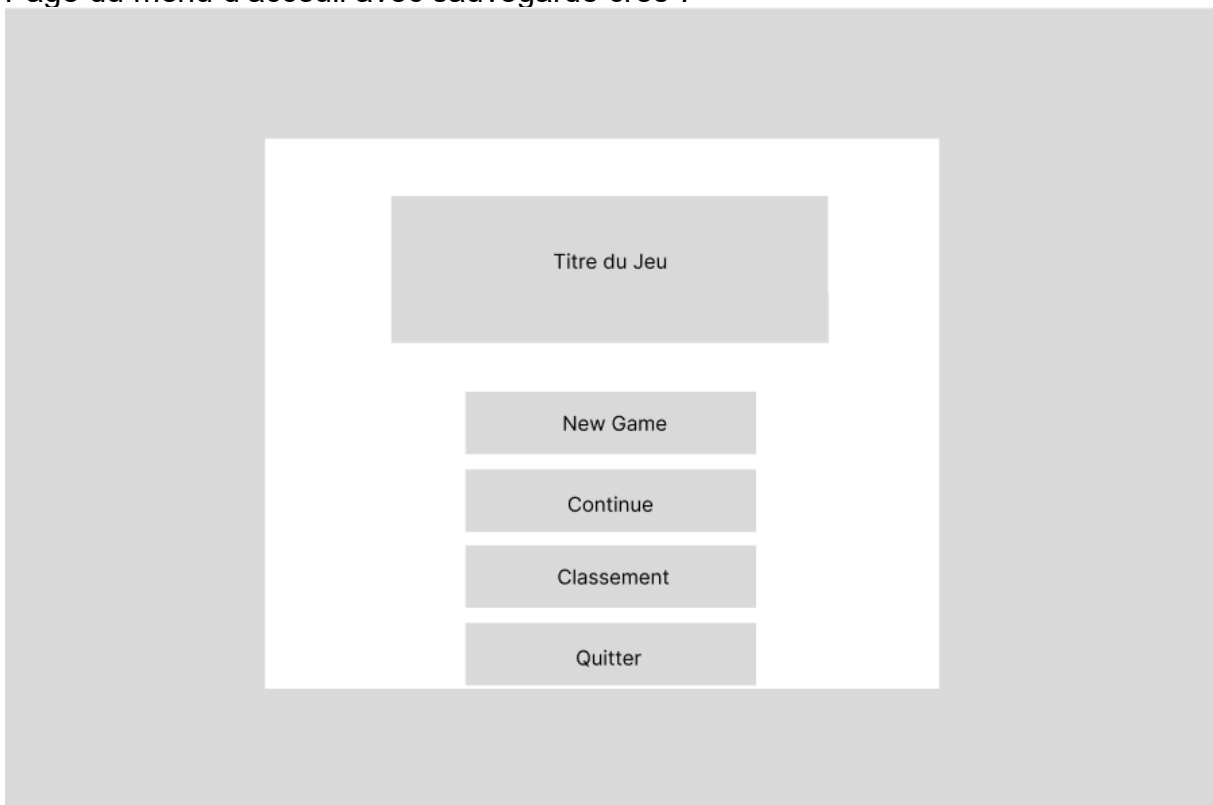


Figure 5: Maquette du jeu avec sauvegarde créée

Page du classement des joueurs en fonction de leur Score :

Classement joueur	
1st	Score 1
2nd	Score 2
3rd	Score 3

Figure 6: Maquette du classement des joueurs

Page du jeu en pleine partie :

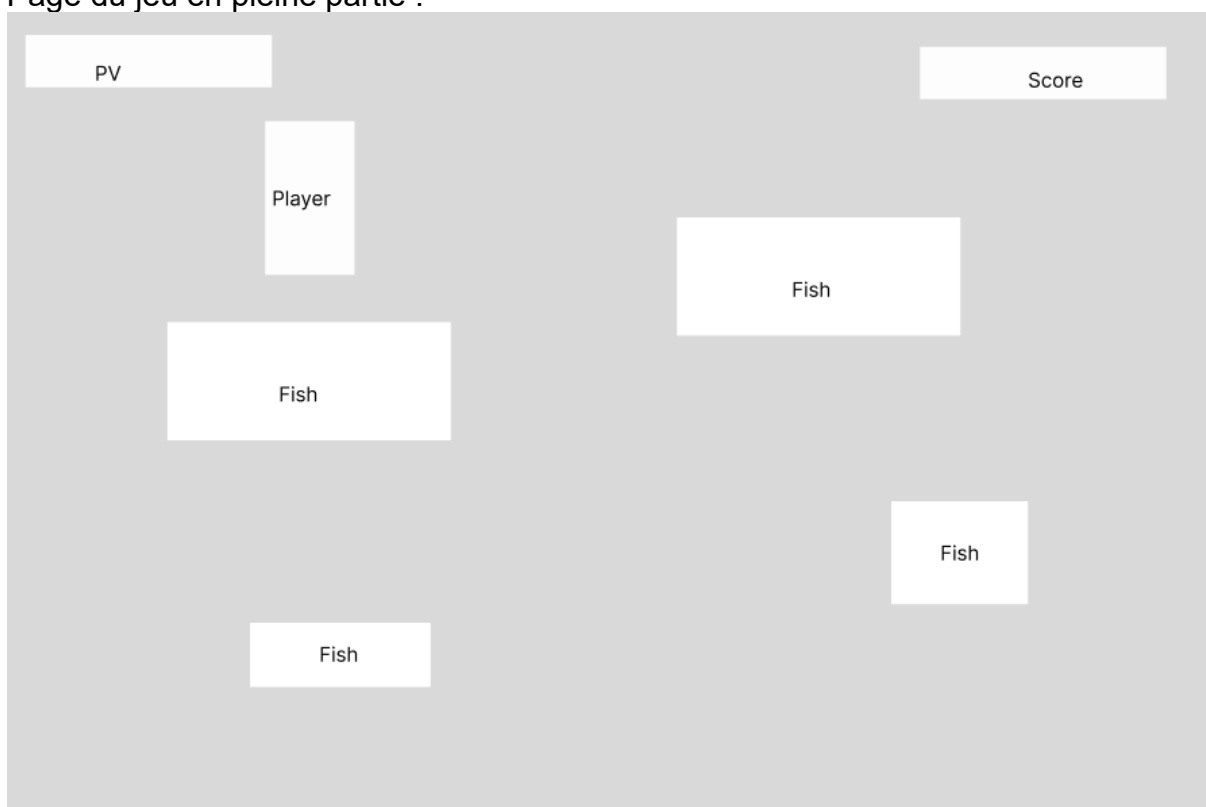


Figure 7: Maquette du jeu en pleine partie

Page plus visuelle du jeu en pleine partie :

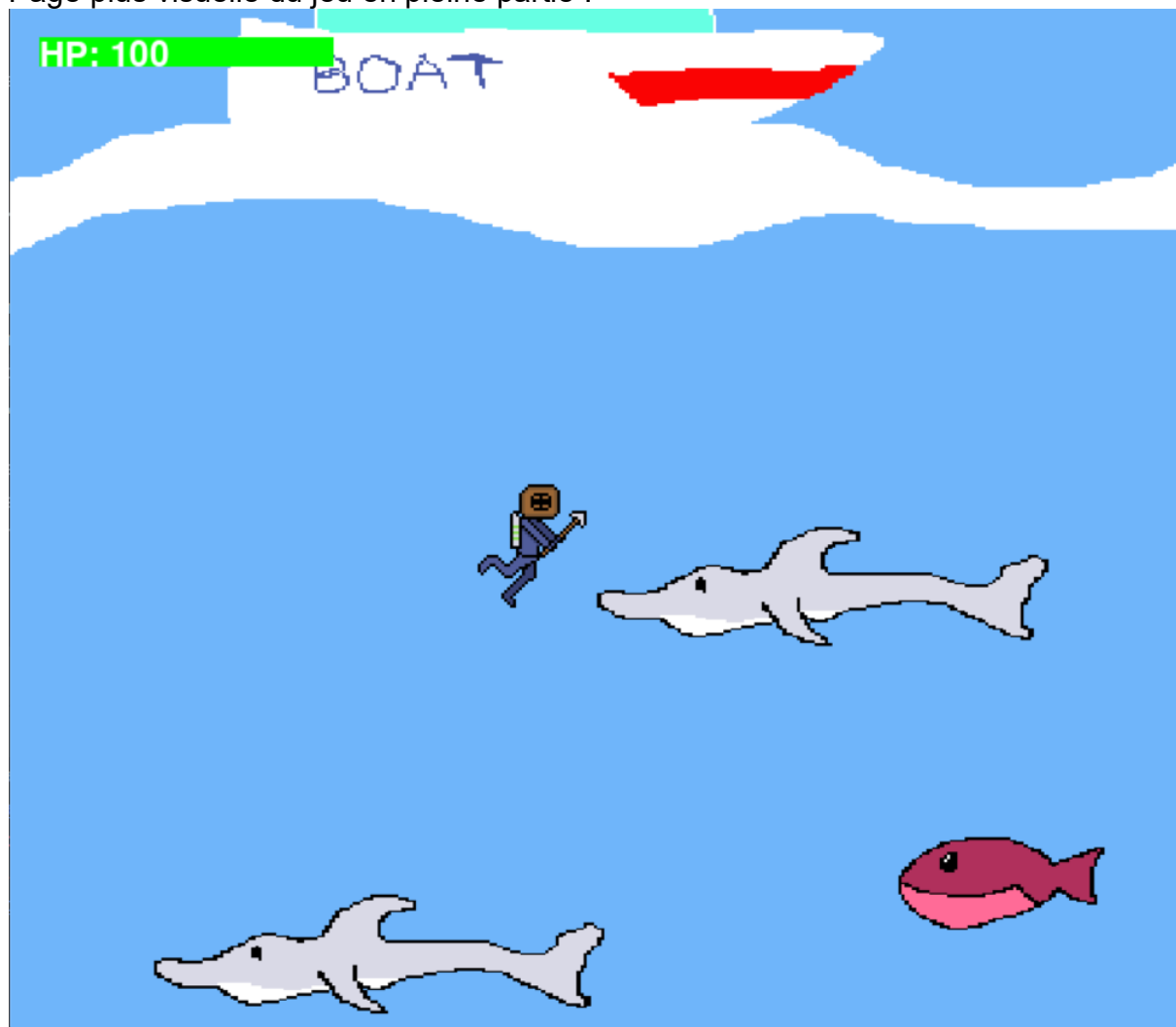


Figure 8: Maquette plus visuel du jeu en pleine partie

2.3 Diagramme de Classe

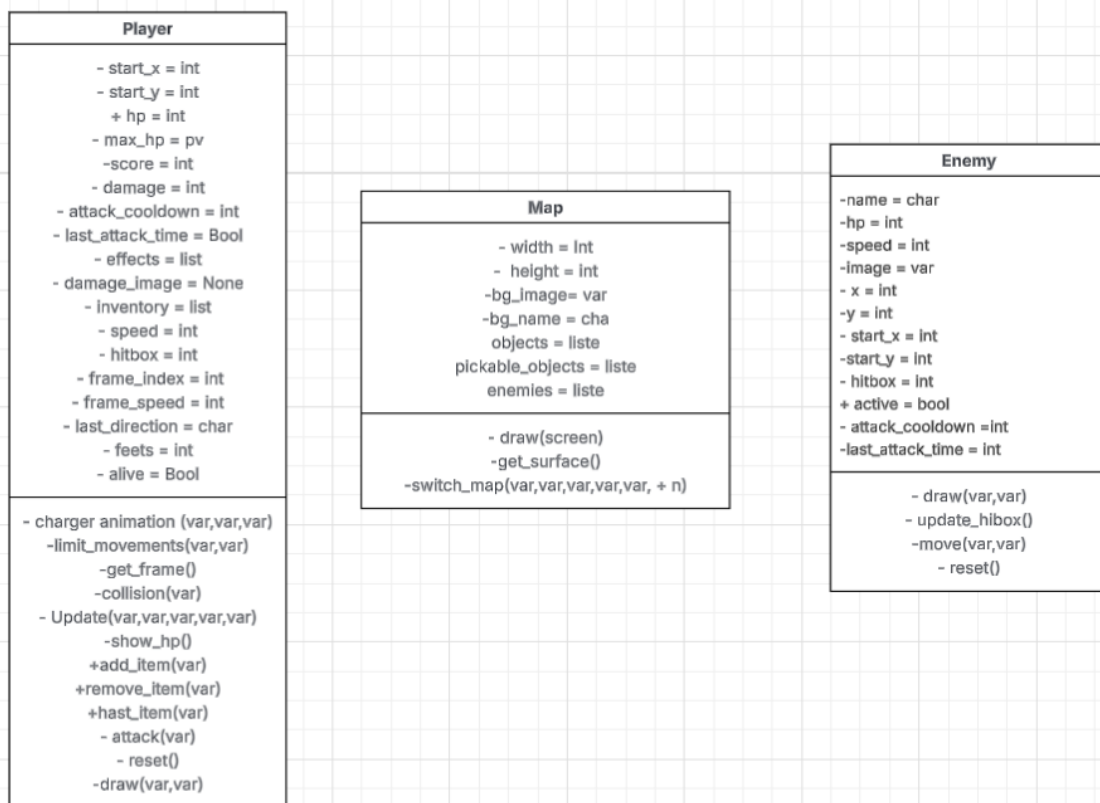


Figure 9: Diagramme de classes 1

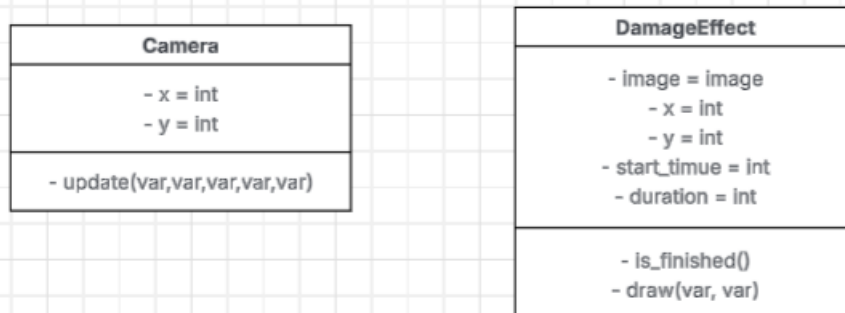


Figure 10: Diagramme de classe 2

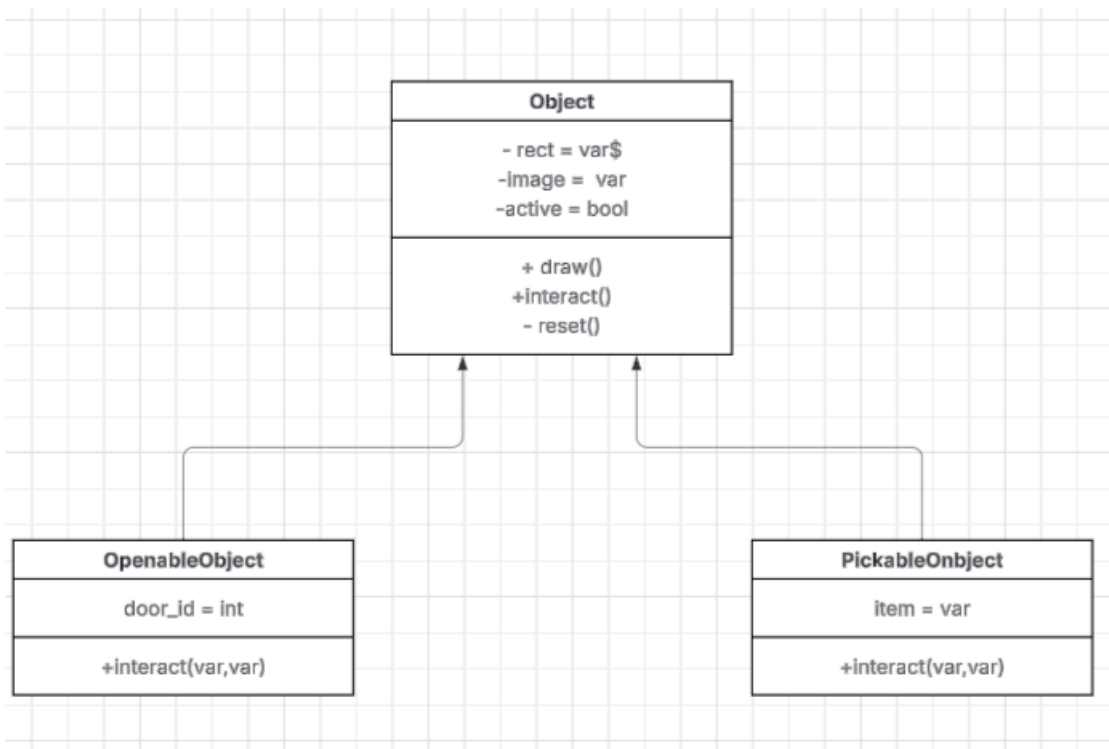


Figure 11: Diagramme de classe 3

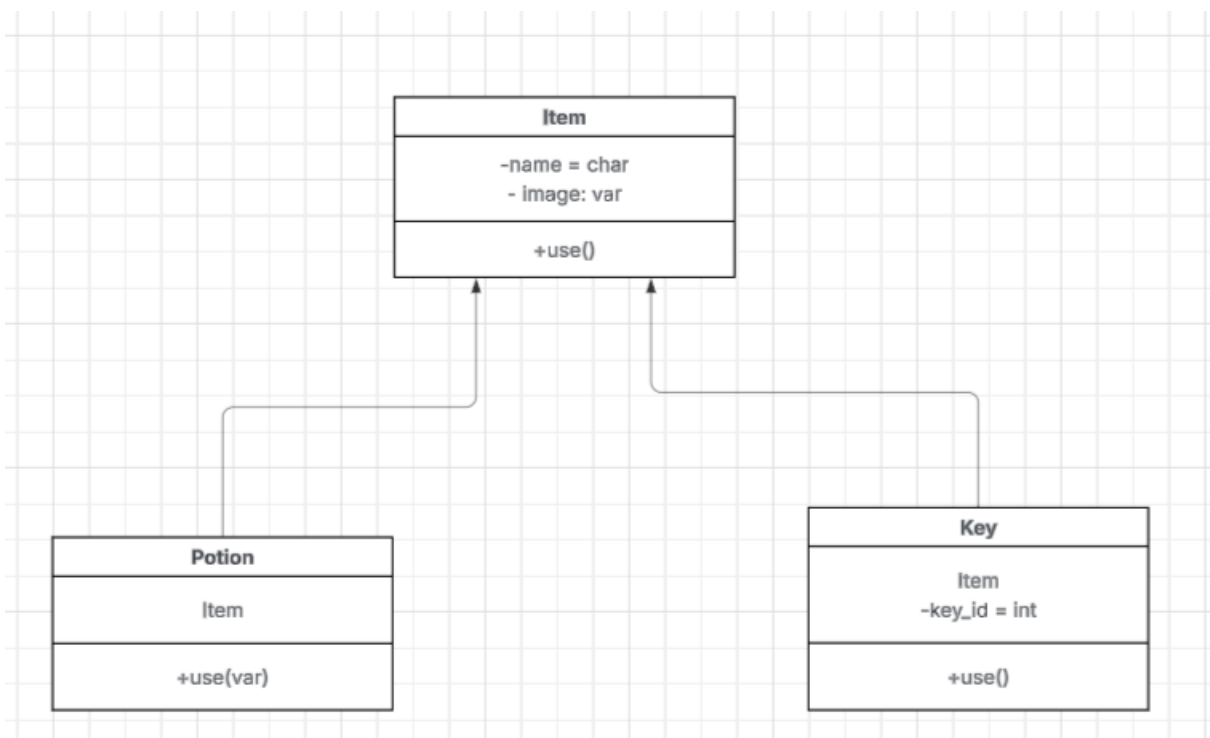


Figure 12: Diagramme de classes 4

Ces diagrammes ci-dessus illustrent la manière dont les différentes classes ont été conçues. On peut voir que la classe **Player** est la plus conséquente, car c'est elle qui est utilisée pour représenter le **Player** et qui par conséquent interagira avec les autres classes afin que le jeu fonctionne correctement. La classe **Enemy** est celle qui ressemble le plus à la classe **Player** car elle possède des attributs similaires à **Player** tels que le `speed` ou les `starts_x` et `start_y`.

Avec autant de similitudes, pourquoi ne pas avoir fait une classe parente regroupant ces attributs similaires puis créer des classes enfants ? Bien que Player et Enemy aient des attributs similaires, ils sont fondamentalement différents car Player est le joueur qu'on incarne et les ennemis sont des bots. Ils ne sont pas comme la classe Potion et la classe Key qui eux sont fondamentalement des Items. Les mettre en classe enfants à la classe Item est donc ici plus judicieux que Player et Enemy, c'est donc pourquoi j'ai choisi de les séparer.

2.4 MCD et MLD

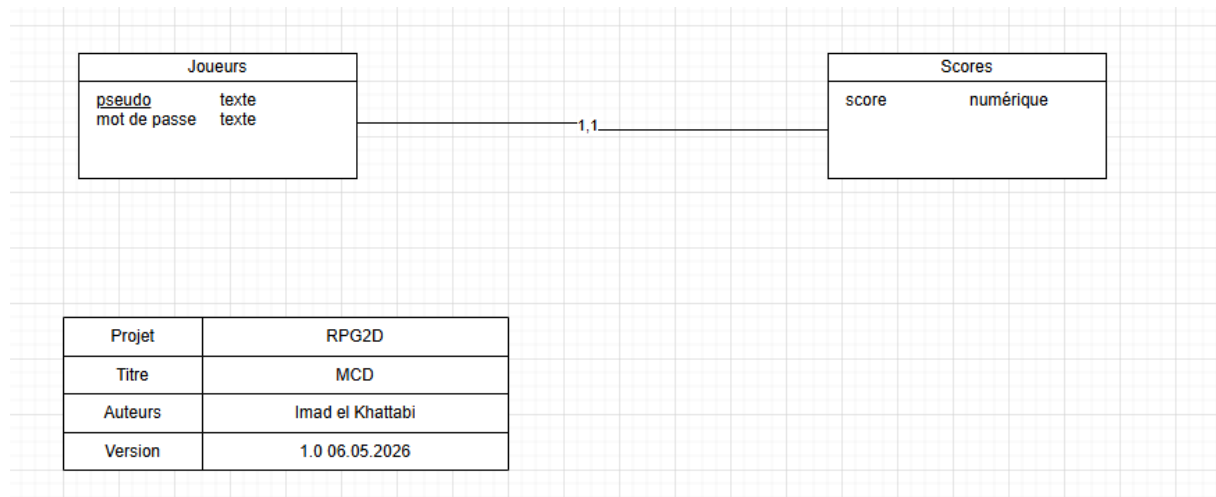


Figure 13: MCD

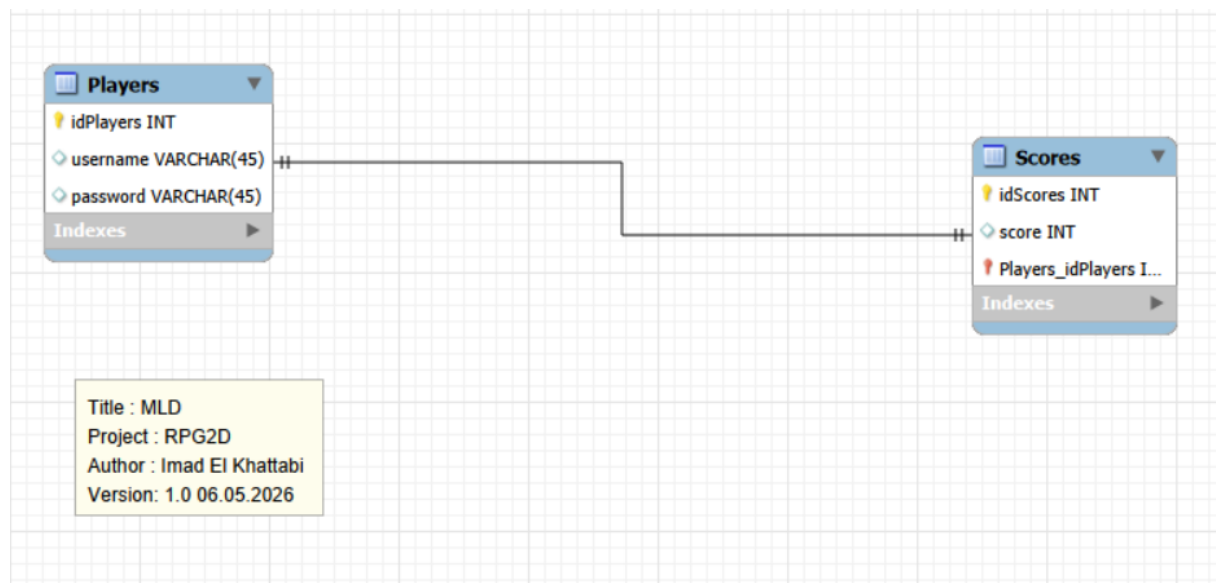


Figure 14: MLD

Le MCD et le MLD sont constitués de deux tables qui serviront pour l'inscription, et la connexion ainsi qu'au pour le classement des joueurs en fonction de leur score. Dans la table *Players* (Joueurs), on retrouve les champs *username* (pseudo), qui est un champ unique afin de différencier les joueurs ainsi qu'un mot de passe non unique afin que tout le monde puisse mettre le mot de passe de son choix pour

mieux protéger les comptes. La table *Score* contient, quant à elle, le champ score ainsi que la clé étrangère de *Players* afin de lier le joueur à son score.

2.5 Choix du Code

Collision joueur (hitbox) :

J'ai décidé de gérer les collisions du joueur avec le système de hitbox qui permettra de gérer plus facilement les attaques de l'ennemi aussi pour avoir un rendu visuel plus réaliste. Comme la hitbox est un rectangle qui se forme seulement autour de la partie dessinée de le sprite lorsque qu'on la configure et non toute le sprite, on peut l'utiliser pour simuler une collision réaliste avec les ennemis. L'autre avantage de la hitbox est que, comme c'est un rectangle le code sera moins lourd que si on faisait calculer l'ordinateur aux pixels près pour avoir un meilleur résultat.

Déplacement de l'ennemi :

En ce qui concerne les déplacements de l'ennemi, j'utilise la librairie de math pour utiliser la fonction « *math.hypot* » qui me permettra de calculer la distance du vecteur entre les deux entités peu importe où elles se trouvent et ainsi pouvoir choisir à partir de quelle distance je veux qu'il se rapproche de la cible.

Changement de map :

Pour le changement de map, le but sera de créer une fonction qui saura reconnaître quelle map est en cours pour qu'elle puisse être liée au bonnes maps. Pour ce faire, j'ai décidé de rentrer le nombre exact de map utilisé en argument pour ensuite créer plusieurs condition if afin de les spécifier pour que le programme puisse savoir quelle map est liée à quelle map afin de sortir la bonne.

2.6 Stratégie de test

En ce qui concerne ma stratégie de test, tous mes tests se feront de manière manuelle à l'exception d'un qui sera automatique qui gérera la version du module Pygame. Pour les autres tests ceux-ci seront manuels car mes fonctions ne peuvent fonctionner que si le jeu n'est pas lancé. Il y aura donc plusieurs tests unitaires pour vérifier le bon fonctionnement de la fonction. Ensuite à la fin un test d'intégration pour tester qu'il n'y a pas de conflits avec les autres fonctions.

Les tests qui m'aideront à la réalisation de ce projet sont :

- Les déplacements des personnages
 - Déplacement vers le haut
 - Déplacement vers le bas
 - Déplacement vers la gauche
 - Déplacement vers la droite
- Barre de points de vie
 - Affichage de la barre de points
 - Prendre des dégâts
 - Mort du joueur

- Attaque du Joueur :
 - Affichage de l'attaque
 - Joueur attaque l'ennemi
 - Joueur tue l'ennemi
- Collision du joueur avec les bords
 - Collision avec le bord supérieur
 - Collision avec le bord inférieur
 - Collision avec le bord de gauche
 - Collision avec le bord de droite
- Collision du joueur avec les objets du monde
 - Le joueur ne traverse pas les objets du monde
- Déplacement ennemi
 - Déplacement vers le haut
 - Déplacement vers le bas
 - Déplacement vers la gauche
 - Déplacement vers la droite
 - L'ennemi ne bouge pas quand il est loin du joueur
- Collision de l'ennemi
 - L'ennemi touche le joueur
 - L'ennemi ne traverse pas le joueur
- Attaque de l'ennemi
 - L'ennemi attaque le joueur
 - L'ennemi tue le joueur
- Changement de map
 - Le perso peut changer de map vers le haut si une map est connecté à cet endroit
 - Le perso peut changer de map vers le bas si une map est connecté à cet endroit
 - Le perso peut changer de map vers la gauche si une map est connecté à cet endroit
 - Le perso peut changer de map vers la droite si une map est connecté à cet endroit
 - Le perso ne peut pas changer de map vers le haut si aucune map n'est connecté à cet endroit
 - Le perso ne peut pas changer de map vers le bas si aucune map n'est connecté à cet endroit
 - Le perso ne peut pas changer de map vers la gauche si aucune map n'est connecté à cet endroit
 - Le perso ne peut pas changer de map vers la droite si aucune map n'est connecté à cet endroit
- Récupération des objets
 - Le joueur récupère l'Item et le stocke dans son inventaire

- Le joueur ne peut récupérer d'Items si l'inventaire a atteint sa capacité maximale
- Système d'inventaire
 - Affichage de l'inventaire
 - Sélectionner un Item de l'inventaire
 - Utiliser une potion de soin avec pas beaucoup de hp
 - Utiliser une potion de soin avec tous ses hp
 - Utiliser une clé sur la bonne porte
 - Utiliser une clé sur la mauvaise porte
- Système d'inscription
 - Inscription correcte
 - Inscription sans son pseudo
 - Inscription sans son mot de passe
 - Inscription sans confirmation de mot de passe
 - Possède déjà un compte
- Système de connexion
 - Connexion correcte
 - Connexion avec mauvais pseudo
 - Connexion avec mauvais mot de passe
 - Connexion avec mauvais pseudo et mot de passe
 - Ne possédant pas de compte
- Système de compte à rebours
 - Affichage du compte à rebours
 - Décompte toute les 1 secondes
- Système de Score
 - Affichage du Score
 - Augmente pour chaque ennemi battu
 - Augmente pour chaque perle récupérée
 - Augmente pour chaque énigme résolue
 - Diminue à chaque fois qu'il prend des dégâts
 - Augmente encore à la fin du jeu s'il reste du temps
 - Score lié au joueur avec la base de données
- Système de points
 - Affichage des points
 - Augmentation du Score
- Système de piège
 - Affichage des pièges
 - Piège blesse le joueur
 - Piège soustrait des points au score
- Système de sauvegarde
 - Sauvegarde automatique à chaque fois que le joueur change de zone
 - Sauvegarde manuelle dans le menu pause

- Sauvegarde de l'ID du joueur
 - Sauvegarde de la zone
 - Sauvegarde de la position du joueur
 - Sauvegarde du nombre de hp du joueur
 - Sauvegarde de l'inventaire du joueur
 - Sauvegarde du Score
 - Sauvegarde de la progression des énigmes
 - Sauvegarde du temps
- Gestion des Menus
 - Menu principal sans sauvegarde précédente
 - Menu principal avec sauvegarde précédente

2.7 Risques techniques

Plusieurs risques peuvent surgir lors de ce projet. Cependant parmi celles-ci, celles qui ont le plus de chance de surgir sont :

- Pertes de données : si on me vole mon pc ou si tout mon projet est corrompu.
- L'incompatibilité sur macOS ou Linux car je code uniquement sur Windows 11
- Le fait que le client ne puisse pas lancer le jeu, car il se pourrait qu'il n'ait pas installé les dépendances requises ou qu'il n'est pas créé la base de données.

2.8 Planification

En ce qui concerne la planification du projet j'ai pris un peu d'avance dans le premier sprint ce qui m'a permis d'approfondir un petit peu sur certaines fonctions comme le système de porte, le changement de map ou la collision des ennemis. Ensuite une fois que je me suis attardé sur certains détails j'ai commencé gentiment à travailler sur les storys du deuxième Sprint.

2.9 Dossier de conception

OS : Windows 11

Logiciel du design des maps, des ennemis, du joueur, des objets et des items : Pixel Studio version : 5.42.0.0

Logiciel de programmation : Jet Brains PyCharm version : 2025.2.1

Langage de programmation : Python : 3.13.9

Documentation : Word Office version : 2602

Dépôt : Git version 2.51.2

Conception des Maquettes : Figma site : <https://www.figma.com>

Conception Diagramme de Classe : https://lucid.app/lucidchart/decc2c58-4c6f-48be-9ac1-b38fe976cb2d/edit?beaconFlowId=A0E0DDBD1EB3FF73&invitationId=inv_8feadd20-0f56-46a1-935a-df7ab8931388&page=HWEp-vi-RSFO#

Conception du MCD : Draw.io version 29.7.9

Conception MLD : MySQL Workbench 8.0 CE

Conception du Script SQL : HeidiSQL version 12.12.0.7122

Corrections des fautes d'orthographe et grammaire : Chat GPT

3 Réalisation

3.1 Dossier de réalisation

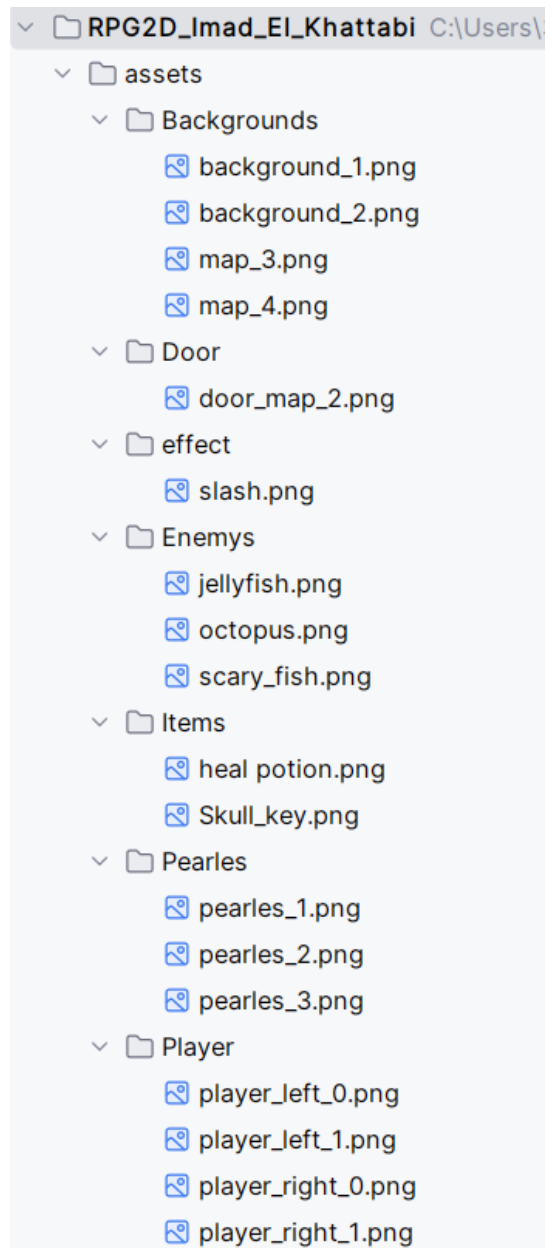


Figure 15: Arborescence des fichiers partie 1

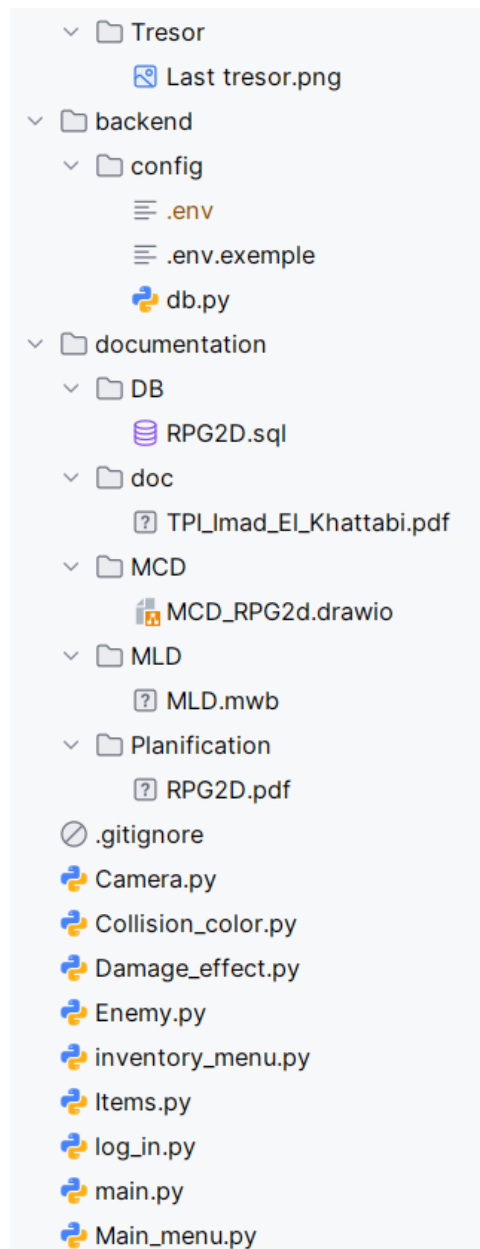


Figure 16: Arborescence des fichiers partie 2

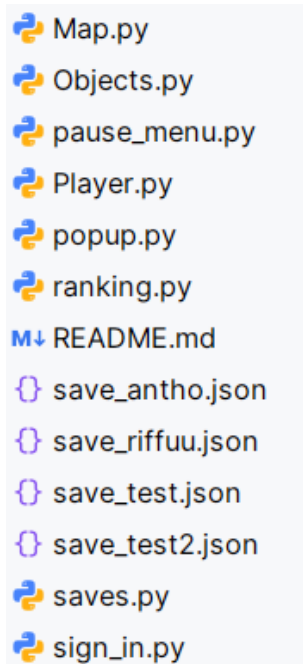


Figure 17: Arborescence des fichiers partie 3

Voici des illustrations de mon dossier réalisation. Tout mes fichiers sont regroupé en fonction de leur type. Nous avons donc le dossier assets qui regroupe tous les Sprites qui sont eux aussi réparties dans des sous-dossiers. Nous avons ensuite le dossier backend qui se chargera des connexion et interactions avec la base de données. Puis viens le dossier documentation qui contient tous les MCD MLD scripte de la base de données ainsi que le rapport du TPI et la planification initial. Le reste des fichiers code sont situés à la racine du dossier RGP2D afin d'aider à la liaison entre les fichiers.

3.2 Description des tests effectués plus résultats des tests

3.2.1 Mouvement du joueur (flèche du haut) :

Description :

Lorsque l'on appuie sur la flèche du haut, le personnage se dirige vers le haut

Test d'acceptation :

Player goes up

Given : The game is started

When : Player press the key arrow up

Then : player goes up

☒ Réussi

Figure 18 : Déplacement vers le haut

3.2.2 Mouvement du joueur (flèche du bas) :

Description :

Lorsque l'on appuie sur la flèche du bas, le joueur se dirige vers le bas

Test d'acceptation :

Player goes down

Given : The game is started

When : Player press the key arrow down

Then : player goes down

☒ Réussi

Figure 19 : Déplacement vers le bas

3.2.3 Mouvement du joueur (flèche de gauche) :

Description :

Lorsque l'on appuie sur la flèche gauche, le joueur se dirige vers la gauche.

Test d'acceptation :

Player goes left

Given : The game is started

When : Player press the key left arrow

Then : player goes left

☒ Réussi

Figure 20 : Déplacement vers la gauche

3.2.4 Mouvement du joueur (flèche de droite) :

Description :

Lorsque l'on appuie sur la flèche du bas, le joueur se dirige vers la droite.

Test d'acceptation :

Player goes right

Given : The game is started

When : Player press the key right arrow

Then : player goes right

☒ Réussi

Figure 21 : Déplacement vers la droite

3.2.5 Affichage de la barre des points de vie :

Description :

Lorsque le jeu est lancé, la barre des points de vie s'affiche en haut à droite de l'écran.

Test d'acceptation :

Show Health Bar

Player is in the game

Player starts playing

Player can see the health bar in the game

☒ Success

Figure 22: Affichage des points de vie

3.2.6 Prendre des dégâts :

Description :

Lorsque le joueur est touché par un ennemi, ses points de vie diminuent.

Test d'acceptation :

Taking damage

Player and enemy are in the game

Player gets hurt by an enemy

Player's health goes down

☒ Success

Figure 23: Dégâts reçus

3.2.7 Mort du joueur :

Description :

Lorsque les points de vie du joueur sont à 0 le joueur disparaît et il est écrit **GAME OVER**.

*Test d'acceptation :***Player dies**

Player is in the game

Player's health reaches 0

Players die *Game Over*

☒ Success

Figure 24: Mort du joueur

3.2.8 Affichage de l'attaque du joueur :

Description :

Lorsque le joueur appuie sur la touche « X » une image d'attaque apparaît puis disparaît.

*Test d'acceptation :***Attack displays**

Player is in the game

Player presses the key X

Player does an attack animation

☒ Success

Figure 25: Affichage attaque du joueur

3.2.9 Joueur attaque l'ennemi :

Description :

Lorsque le joueur est proche du joueur et qu'il appuie sur X l'ennemi prend des dégâts.

*Test d'acceptation :***Player attacks enemy**

Player and enemy are in the game

Player presses the key X and touches the enemy

Enemy gets hurt

☒ Success

Figure 26: Joueur attaque l'ennemi

3.2.10 Joueur tue l'ennemi :

Description :

Lorsque le joueur attaque l'ennemi il le tue et l'ennemi disparaît.

Test d'acceptation :

Player killed enemy

Player and enemy are in the game

Player presses the key X and kills the enemy

Enemy disappears and doesn't hurt player anymore

☒ Success

Figure 27: Joueur tue l'ennemi

3.2.11 Collision du joueur avec le bord supérieur

Description :

Lorsque que le joueur touche le bord supérieur de l'écran il est bloqué.

Test d'acceptation :

Collision with the border top

Player is in the game

He touches the border top of the screen

He cannot go through

☒ Success

Figure 28: Collision du joueur avec le bord supérieur

3.2.12 Collision du joueur avec le bord inferieur

Description :

Lorsque que le joueur touche le bord inferieur de l'écran il est bloqué.

*Test d'acceptation :***Collision with the bottom border**

Player is in the game

He touches the border bottom of the screen

He cannot go through

☒ Success

Figure 29: Collision du joueur avec le bord inferieur

3.2.13 Collision du joueur avec le bord de gauche

Description :

Lorsque que le joueur touche le bord gauche de l'écran il est bloqué.

*Test d'acceptation :***Collision with the left border**

Player is in the game

He touches the left border of the screen

He cannot go through

☒ Success

Figure 30: Collision du joueur avec le bord gauche

3.2.14 Collision du joueur avec le bord de droite

Description :

Lorsque que le joueur touche le bord droit de l'écran il est bloqué.

*Test d'acceptation :***Collision with the right border**

Player is in the game

He touches the right border of the screen

He cannot go through

☒ Success

Figure 31: Collision du joueur avec le bord droit

3.2.15 Collision du joueur avec les objets

Description :

Lorsque que le joueur touche un objet, il est bloqué.

*Test d'acceptation :***Collision with objects:**

Player is in the game

He touches any objects in the game

He cannot go through it

☒ Success

Figure 32: Collision du joueur avec les objets

3.2.16 Déplacement de l'ennemi vers le haut :

Description :

Lorsque que le joueur est au-dessus de l'ennemi et proche l'ennemi se déplace vers le haut.

*Test d'acceptation :***Enemy goes up**

Player and enemy are in the game

Player is above and close to the enemy

The enemy start chasing him by going up

☒ Success

Figure 33: Déplacement de l'ennemi vers le haut

3.2.17 Déplacement de l'ennemi vers le bas :

Description :

Lorsque que le joueur est en dessous de l'ennemi et proche l'ennemi se déplace vers le bas.

*Test d'acceptation :***Enemy goes down**

Player and enemy are in the game

Player is under and close to the enemy

The enemy start chasing him by going down

☒ Success

Figure 34: Déplacement de l'ennemi vers le bas

3.2.18 Déplacement de l'ennemi vers la gauche :

Description :

Lorsque que le joueur est à gauche de l'ennemi et proche l'ennemi se déplace vers la gauche.

Test d'acceptation :

Enemy goes left

Player and enemy are in the game

Player is close and to the left of the enemy

The enemy start chasing him by going to the left

☒ Success

Figure 35: Déplacement de l'ennemi vers la gauche

3.2.19 Déplacement de l'ennemi vers la droite :

Description :

Lorsque que le joueur est à droite de l'ennemi et proche l'ennemi se déplace vers la droite.

Test d'acceptation :

Enemy goes right

Player and enemy are in the game

Player is close and to the right of the enemy

The enemy start chasing him by going right

☒ Success

Figure 36: Déplacement de l'ennemi vers la droite

3.2.20 Ennemi est loin du joueur :

Description :

Lorsque que le joueur ne se trouve pas dans le champ de l'ennemi, l'ennemi ne bouge pas.

*Test d'acceptation :***Player is far from Enemy**

Player and enemy are in the game

Player is far from enemy

The enemy don't chase the player

☒ Success

Figure 37: Ennemi loin du joueur

3.2.21 Ennemi touche le joueur :

Description :

Lorsque que l'ennemi touche le joueur, il le blesse et ne le traverse pas.

*Test d'acceptation :***Enemy touches player**

Player and Enemy are in the Game

Enemy goes towards the player

Enemy hurts Player and doesn't go through it

☒ Success

Figure 38: Collision de l'ennemi avec le joueur

3.2.22 Ennemi touche les objets :

Description :

Lorsque que l'ennemi se dirige vers un objet il est bloqué par celui-ci et ne peut pas aller plus loin.

*Test d'acceptation :***Enemy touches map objects**

Enemy is in the Game

Enemy goes towards an object

Enemy gets stuck and can't go any further

☒ Success

Figure 39: Collision de l'ennemi avec les objets

3.2.23 Ennemi attaque le joueur :

Description :

Lorsque que l'ennemi touche le joueur, le joueur perd des dégâts.

Test d'acceptation :

Enemy touches Player

Player and enemy are in the game

Enemy touches player

Player start taking damage

☒ Success

Figure 40: Ennemi attaque le joueur

3.2.24 Ennemi tue le joueur :

Description :

Lorsque que l'ennemi touche le joueur, il le blesse jusqu'à le tuer et le joueur disparaît et il est écrit *GAME OVER*.

Test d'acceptation :

Player dead

Player and Enemy are in the game

Enemy touches player and killed the player

Player disappear the hp show 0 and there's a big message that's says : *GAME OVER*

☒ Success

Figure 41: Ennemi tue le joueur

3.2.25 Changement de map du haut vers le bas :

Description :

Lorsque que le joueur se dirige vers le haut de la map et qu'une map est lié en haut, il se retrouve tout en bas de la map suivante.

*Test d'acceptation :***Link up to down**

Two background connected by sides

Player go to the top side of one background

He appears in the next background to the bottom

☒ Success

Figure 42: Changement de map du haut vers le bas

3.2.26 Changement de map du bas vers le haut :

Description :

Lorsque que le joueur se dirige vers le bas de la map et qu'une map est lié en bas, il se retrouve tout en haut de la map suivante.

*Test d'acceptation :***Link down to up**

Two background connected by sides

Player go to the bottom side of one background

He appears in the next background to the top

☒ Success

Figure 43: Changement de map du bas vers le haut

3.2.27 Changement de map de la gauche vers la droite :

Description :

Lorsque que le joueur se dirige à gauche de la map et qu'une map est lié à gauche, il se retrouve tout à droite de la map suivante.

*Test d'acceptation :***Link left to right**

Two background connected by sides

Player go the very left side of one background

He appears in the next background to the right

☒ Success

Figure 44: Changement de map de la gauche vers la droite

3.2.28 Changement de map de la droite vers la gauche :

Description :

Lorsque que le joueur se dirige à droite de la map et qu'une map est lié à droite, il se retrouve tout à gauche de la map suivante.

Test d'acceptation :

Two background connected by sides

Player go the very right side of one background

He appears in the next background to the left

☒ Success

Figure 45: Changement de map de la droite vers la gauche

3.2.29 Changement de map pas de lien vers le haut :

Description :

Lorsque que le joueur se dirige à droite de la map et qu'aucun map est lié en haut, il se retrouve bloqué au bord.

Test d'acceptation :

Not linked to the top

A background is not connected to the top

Player go to the top side of the background

He still next to the top border and the background doesn't change

☒ Success

Figure 46: Changement de map pas le lien vers le haut

3.2.30 Changement de map pas de lien vers le bas :

Description :

Lorsque que le joueur se dirige à droite de la map et qu'aucun map est lié en haut, il se retrouve bloqué au bord.

*Test d'acceptation :***Not linked to the bottom**

A background is not connected to the bottom

Player go to the bottom side of the background

He still next to the bottom border and the background doesn't change

☒ Success

Figure 47: Changement de map pas le lien vers le bas

3.2.31 Changement de map pas de lien vers la gauche :

Description :

Lorsque que le joueur se dirige à droite de la map et qu'aucun map est lié en haut, il se retrouve bloqué au bord.

*Test d'acceptation :***Not linked to the left**

A background is not connected to the left

Player go to the left side of the background

He still next to the left border and the background doesn't change

☒ Success

Figure 48: Changement de map pas le lien vers la gauche

3.2.32 Changement de map pas de lien vers la droite :

Description :

Lorsque que le joueur se dirige à droite de la map et qu'aucun map est lié en haut, il se retrouve bloqué au bord.

*Test d'acceptation :***Not linked to the right**

A background is not connected to the right

Player go to the right side of the background

He still next to the right border and the background doesn't change

☒ Success

Figure 49: Changement de map pas le lien vers la droite

3.2.33 Récupération de l'item et stockage dans l'inventaire :

Description :

Lorsque le joueur se trouve à proximité d'un item et que son Inventaire n'est pas rempli, celui-ci peut le récupérer et le stocker dans son inventaire en appuyant sur la touche « E ».

*Test d'acceptation :***Player takes Item**

Player is in the game

He's next to an Item and presses the Key "E"

The Item dissapears and appears in his inventory

☒ Success

Figure 50: Récupération de l'Item et stockage dans l'inventaire

3.2.34 Incapacité du joueur à récupérer un item (Inventaire rempli) :

Description :

Lorsque le joueur se trouve à proximité d'un item et que son Inventaire est pas rempli, celui-ci ne peut pas le récupérer et le stocker dans son inventaire en appuyant sur la touche « E ».

*Test d'acceptation :***Inventory full**

Player is in the game with his inventory full

He's next to an Item and presses the Key "E"

The Item doesn't dissapears and isn't stock in his inventory

☒ Success

Figure 51: Incapacité du joueur à récupérer un item (Inventaire rempli)

3.2.35 Affichage de l'inventaire :

Description :

Lorsque que le joueur appuie sur la touche « I » il accède à l'inventaire

*Test d'acceptation :***Inventory opens**

Player is in the game

Player press the key "I"

Inventory opens

☒ Success

Figure 52: Affichage de l'inventaire

3.2.36 Sélectionner un Item de l'inventaire :

Description :

Lorsque que le joueur se trouve dans l'inventaire il peut sélectionner et choisir l'Item qu'il voudra utiliser avec les touches fléchées.

*Test d'acceptation :***Select an Item**

Player is in the Inventory with a lot of Items in it

Player press the keys Up ou Down

He can choose an Item

☒ Success

Figure 53: Sélectionner un Item dans l'inventaire

3.2.37 Utiliser une potion de soin avec peu de points de vie :

Description :

Lorsque que le joueur possède peut d'hp, qu'il va dans l'inventaire et qu'il sélectionne la potion de soin, celui-ci peut retrouver une partie de ses points de vie en appuyant sur la touche « E ». La potion disparaît après l'utilisation.

Test d'acceptation :

Using an Potion with low hp

Player is in the Inventory with a Potion

Player press the key E

Player gets more Hp and the potion dissapears

☒ Success

Figure 54: Utiliser une potion de soin avec peu d'hp

3.2.38 Utiliser une potion de soin avec tous ses points de vie :

Description :

Lorsque que le joueur possède toujours tous ses points de vie, qu'il va dans l'inventaire et qu'il sélectionne la potion de soin, celui-ci peut consommer la potion sans que cela ne lui procure plus d'hp en appuyant sur la touche « E ». La potion disparaît après l'utilisation.

Test d'acceptation :

Using an Potion with max hp

Player is in the Inventory with a Potion

Player press the key E

Player doesn't get more Hp and the potion dissapears

☒ Success

Figure 55: Utiliser une potion de soin avec tous points de vie

3.2.39 Utiliser la clé sur la bonne porte :

Description :

Lorsque que le joueur possède une clé et se trouve à côté de la bonne porte, il peut l'ouvrir en appuyant sur la touche « E » ce qui fera disparaître la clé de l'inventaire ainsi que la porte qui bloquait le passage.

*Test d'acceptation :***Using a key with the right door**

Player is in the Inventory with a Key next to the right door

Player press the key E

The door and the key dissapear

☒ Success

Figure 56: Utiliser la clé sur la bonne porte

3.2.40 Inscription correcte

Description :

Lorsque que le joueur crée son compte en mettant son pseudo, son mot de passe et en confirmant son mot de passe il peut appuyer sur le bouton et son compte est créé.

*Test d'acceptation :***Sign in correctly**

User launched the game

He enter his username and his password, confirms it and click on sign in button

His account is created and he can play the game

☒ Success

Figure 57: Inscription correcte

3.2.41 Le joueur doit obligatoirement mettre son pseudo

Description :

Lorsque que le joueur rempli les champs sauf celui du pseudo, il y a une erreur, et son compte n'est pas créé.

*Test d'acceptation :***Sign in without username**

User launched the game

He enter his password, confirms it and click on sign in button

His account is not created and a message appears saying *Champs vides*

 Success

Figure 58: Inscription sans pseudo

3.2.42 Le joueur doit obligatoirement mettre son mot de passe :

Description :

Lorsque que le joueur entre seulement son pseudo il y a une erreur et son compte n'est pas créé.

*Test d'acceptation :***Sign in without password**

User launched the game

He enter his username and click on sign in button

His account is not created and a message appears saying *Champs vides*

 Success

Figure 59: Inscription sans mot de passe

3.2.43 Ne confirme pas le mot de passe

Description :

Lorsque que le joueur entre son pseudo son mot de passe mais le confirme pas, une erreur s'affiche et son compte n'est pas créé.

*Test d'acceptation :***Sign in with not same password**

User launched the game

He enter his username , his password and enter a different password in confirme password click on sign in button

His account is not created and a message appears saying *Les mots de passes ne correspondent pas*

☒ Success

Figure 60: Inscription sans confirmation de mot de passe

3.2.44 Possède déjà un compte

Description :

Lorsque que le joueur possède déjà un compte il appuie sur le bouton « Already have an account » et atterri sur la page de connexion.

*Test d'acceptation :***Already have an account**

User launched the game

He clicks on the *Already have an account* button

He's now in the login page

☒ Success

Figure 61: Possédant déjà un compte

3.2.45 Connexion correcte

Description :

Lorsque que le joueur possède déjà un compte il se connecte sur la page de connexion avec son pseudo et son mot de passe

*Test d'acceptation :***Log In correctly**

User is in the login page

He log in correctly

He can play the game

☒ Success

Figure 62: Connexion correcte

3.2.46 Connexion avec mauvais pseudo

Description :

Lorsque que le joueur entre seulement son mot de passe, une erreur s'affiche et il ne peut pas jouer au jeu.

Test d'acceptation :

Log In username wrong

User is in the login page

He log in with a wrong username

He cannot play the game and a message appears saying *Utilisateur ou mot de passe incorrecte*

☒ Success

Figure 63: Connexion avec mauvais pseudo

3.2.47 Connexion avec mauvais mot de passe

Description :

Lorsque que le joueur entre son pseudo mais pas son mot de passe, une erreur s'affiche et il ne peut pas jouer au jeu.

Test d'acceptation :

Log In password wrong

User is in the login page

He log in with a wrong username

He cannot play the game and a message appears saying *Utilisateur ou mot de passe incorrecte*

☒ Success

Figure 64: Connexion avec mauvais mot de passe

3.2.48 Connexion avec mauvais pseudo et mot de passe

Description :

Lorsque que le joueur entre un mauvais pseudo et un mauvais pseudo, une erreur s'affiche et il ne peut jouer.

*Test d'acceptation :***Log In password and username wrong**

User is in the login page

He log in with a wrong username

He cannot play the game and a message appears saying *Nom d'utilisateur ou mot de passe incorrecte*

 Success

Figure 65: Connexion avec mauvais pseudo et mot de passe

3.2.49 Ne possédant pas de compte

Description :

Lorsque que le joueur ne possède pas de compte il peut cliquer sur le bouton servant à le rediriger vers la page d'inscription.

*Test d'acceptation :***Doesn't have an account**

User is in the login page

He clicks on the *Don't have an account* button

He is now in the register page

 Success

Figure 66: Possédant pas de compte

3.2.50 Afficher le Timer

Description :

Lorsque que le joueur joue il y a le timer qui est afficher en haut à droite de l'écran

*Test d'acceptation :***Timer Display**

Player is in the game

He looks in the top right corner of the screen

He sees the Timer

 Success

Figure 67: Affichage du Timer

3.2.51 Décompte du Timer

Description :

Lorsque que le joueur ne possède pas de compte il peut cliquer sur le bouton servant à le rediriger vers la page d'inscription.

Test d'acceptation :

Timer countdown

Player is in the game

He looks in the top right corner of the screen

He sees the Timer counting down every seconds

☒ Success

Figure 68: Décompte du Timer

3.2.52 Affichage du Score

Description :

Lorsque le joueur est dans le jeu il peut apercevoir son Score en tout temps en haut à gauche de l'écran

Test d'acceptation :

Score Display in game

Player is in the game

He looks under the health bar

He sees his Score

☒ Success

Figure 69: Affichage du Score

3.2.53 Augmentation du score pour chaque ennemi battu

Description :

Lorsque le joueur bat un ennemi, son score augmente.

*Test d'acceptation :***Earn points by enemies**

Player is in the game

He killed an enemy

He earns 100 points

☒ Success

Figure 70: Points gagner pour chaque ennemi battu

3.2.54 Augmentation du score pour chaque perle récupérée

Description :

Lorsque le joueur récupère une perle, son score augmente.

*Tests d'acceptations :***Points collected**

Player is in the game

He touches a pearl

He earns some points in the score and the pearl disappears

☒ Success

Figure 71: Augmentation du score pour chaque perle récupérée

3.2.55 Augmentation du score pour chaque énigme résolue

Description :

Lorsque le joueur résout une énigme, son score augmente.

*Test d'acceptation :***Earn points by unlocking doors**

Player is in the game

He unlocks a door

He earns 50 points

☒ Success

Figure 72: Augmentation du score pour chaque énigme résolue

3.2.56 Diminution du score à chaque fois que le joueur prend des dégâts

Description :

Lorsque le joueur subi des dégâts son score diminue.

Test d'acceptation :

Earn points by unlocking doors

Player is in the game

He unlocks a door

He earns 50 points

☒ Success

Figure 73: Points gagné pour chaque énigme résolue

3.2.57 Augmentation du Score avec le temps restant à la fin du jeu

Description :

Lorsque le joueur termine le jeu et que le compte à rebours n'est pas terminé, celui-ci remporte comme points le nombre de secondes qu'il lui reste.

Test d'acceptation :

Score bonus time

Player is in the game

He claims the final tresor

The time left is added to score and the player sees his final score and a message saying *VICTORY*

☒ Success

Figure 74: Score bonus

3.2.58 Score insérer dans la base de données lorsqu'il meurt

Description :

Lorsque le joueur meurt et qu'il n'avait pas de score sauvegardé avant, son score sauvegardé dans la base de données.

*Test d'acceptation :***Score insert in database when he dies**

Player is in the game

He dies

His score is adding in the database

☒ Success

Figure 75: Insertion du score dans la base de données lorsque le joueur meurt

3.2.59 Score insérer dans la base de données lorsqu'il gagne

Description :

Lorsque le joueur gagne la partie et qu'il n'avait pas de score sauvegardé avant, son score sauvegardé dans la base de données.

*Test d'acceptation :***Score insert in database when he wins**

Player is in the game

He wins

His score is adding in the database

☒ Success

Figure 76: Insertion du score dans la base de données lorsque le joueur gagne

3.2.60 Score non mis à jour s'il est inférieur à celui enregistré

Description :

Lorsque le joueur gagne la partie ou meurt et qu'il avait déjà sauvegardé un score auparavant et que le score qu'il vient de réaliser est inférieur à celui enregistré. Il n'est pas sauvegardé dans la base de données.

*Test d'acceptation :***Score doesn't update when it's lower than before when player dies**

Player is in the game

He dies with a score lower than before

His score isn't updating in the database

☒ Success

Score doesn't update when it's lower than before when player wins

Player is in the game

He wins with a score lower than before

His score isn't updating in the database

Figure 77: Tests des deux cas où le score n'est pas mis à jour

3.2.61 Score mis à jour s'il est supérieur à celui enregistré

Description :

Lorsque le joueur gagne la partie ou meurt et qu'il avait déjà sauvegardé un score auparavant et que le score qu'il vient de réaliser est supérieur à celui enregistré. Il est sauvegardé dans la base de données.

*Test d'acceptation :***Score updates when player dies**

Player is in the game

He dies with a better score than before

His score is updating in the database

☒ Success

Score updates when player wins

Player is in the game

He wins with a better score than before

His score is updating in the database

☒ Success

Figure 78: Tests des deux cas où le score est mis à jour

3.2.62 Affichage des points

Description :

Lorsque le joueur est en pleine partie, il peut trouver dans la map des perles servant de point.

Test d'acceptation :

Points Display

Player is in the game

He looks around the map

He sees the points to collect

☒ Success

Figure 79: Affichage des points

3.2.63 Augmentation des points

Description :

Lorsque le joueur s'approche des perles, il peut les collecter pour recevoir plus de points. Les perles collectées disparaissent par la suite.

Test d'acceptation :

Points collected

Player is in the game

He touches a pearl

He earns some points in the score and the pearl disappears

☒ Success

Figure 80: Augmentation de points

3.2.64 Affichage des pièges

Description :

Lorsque le joueur est en pleine partie, il peut trouver des pièges dans la map.

Test d'acceptation :

Traps Display

Player is in the game

He looks around the map

He sees the traps

☒ Success

Figure 81: Affichage des pièges

3.2.65 Piège blessant le joueur

Description :

Lorsque le joueur s'approche des pièges, ceux-ci blessent le joueur.

Test d'acceptation :

Traps giving damage

Player is in the game

He touches a trap

He takes damage

☒ Success

Figure 82: Piège blessant le joueur

3.2.66 Piège diminuant le score

Description :

Lorsque le joueur s'approche des pièges, ceux-ci diminuent le score du joueur.

Test d'acceptation :

Traps subtracts score

Player is in the game

He touches a trap

He loses some points

☒ Success

Figure 83: Piège diminuant le score

3.2.67 Sauvegarde automatique à chaque changement de zone

Description :

Lorsque le joueur change de zone, le jeu se sauvegarde automatiquement.

Test d'acceptation :

Auto-Save

Player is playing

Player goes to another map

The game is saved automatically

☒ Success

Figure 84: Sauvegarde automatique

3.2.68 Sauvegarde manuelle dans le menu pause

Description :

Lorsque le joueur se trouve sur le menu pause et qu'il clique sur le bouton Sauvegarder, il sauvegarde le jeu.

Test d'acceptation :

Save Game button

Player is in the game

Player put the game on pause and click on the *Sauvegarder* button

The game is saved

☒ Success

Figure 85 Sauvegarde manuelle

3.2.69 Sauvegarde de l'ID du joueur

Description :

Lorsque le jeu se sauvegarde, il sauvegarde l'ID du joueur.

Test d'acceptation :

Players ID saved

Player is playing

Player he saves his game

His ID is saved

 Success

Figure 86: ID joueur sauvegardé

3.2.70 Sauvegarde de la zone

Description :

Lorsque le jeu se sauvegarde, il sauvegarde la zone actuelle du joueur.

Test d'acceptation :

current map saved

Player is playing

Player he saves his game

The current map is saved

 Success

Figure 87: Zone actuelle sauvegardé

3.2.71 Sauvegarde de la position du joueur

Description :

Lorsque le jeu se sauvegarde, il sauvegarde la position actuelle du joueur.

Test d'acceptation :

Players position saved

Player is playing

Player he saves his game

His position is saved

 Success

Figure 88: Position du joueur sauvegardé

3.2.72 Sauvegarde du nombre de hp du joueur

Description :

Lorsque le jeu se sauvegarde, il sauvegarde les points de vie actuels du joueur.

Test d'acceptation :

Players hp saved

Player is playing

Player he saves his game

His hp is saved

 Success

Figure 89: Points de vie du joueur sauvegardé

3.2.73 Sauvegarde de l'inventaire du joueur

Description :

Lorsque le jeu se sauvegarde, il sauvegarde l'état actuel de l'inventaire du joueur.

Test d'acceptation :

Players inventory saved

Player is playing

Player he saves his game

His inventory is saved

 Success

Figure 90: Inventaire du joueur sauvegardé

3.2.74 Sauvegarde du Score

Description :

Lorsque le jeu se sauvegarde, il sauvegarde le Score actuel du joueur.

Test d'acceptation :

Score saved

Player is playing

Player he saves his game

The Score is saved

 Success

Figure 91: Score sauvegardé

3.2.75 Sauvegarde des énigmes

Description :

Lorsque le jeu se sauvegarde, il sauvegarde l'état des énigmes.

Test d'acceptation :

Riddles saved

Player is playing

Player he saves his game

The Riddles are saved

 Success

Figure 92: Énigmes sauvegardées

3.2.76 Sauvegarde du temps

Description :

Lorsque le jeu se sauvegarde, il sauvegarde le temps restant du joueur.

Test d'acceptation :

Time saved

Player is playing

Player he saves his game

The Time is saved

 Success

Figure 93: Sauvegarde du temps

3.2.77 Charger une partie

Description :

Lorsque le joueur possède déjà une sauvegarde et qu'il appuie dans le menu principal sur le bouton *Continue*, il reprend sa partie précédente.

Test d'acceptation :

Continue button

Player is in the Start Menu

Player click on the *Continuer* button

Player play with his saved file

☒ Success

Figure 94: Charger une partie

3.2.78 Menu principal sans sauvegarde précédente

Description :

Lorsque le jeu ne possède aucune sauvegarde, il ouvre le menu principal sans possibilité de continuer une partie.

Test d'acceptation :

Start Menu without Save Game

Player log in

The start Menu shows up

The option *Continue* does not appear with the options *New Game* and *Quit*

☒ Success

Figure 95: Menu principal sans sauvegarde précédente

3.2.79 Menu principal avec sauvegarde précédente

Description :

Lorsque le jeu possède une sauvegarde, il ouvre le menu principal avec la possibilité de continuer sa partie précédente partie.

*Test d'acceptation :***Start Menu with a Save Game**

Player log in

The start Menu shows up

The option *Continue* appears with the options *New Game* and *Quit*

☒ Success

Figure 96: Menu principal avec sauvegarde précédente

3.2.80 Menu pause

Description :

Lorsque le joueur est en pleine partie et qu'il appuie sur la touche *Esc*, le menu pause s'affiche.

*Test d'acceptation :***Pause Menu**

Player is in the Game

Player presses the key *Esc*

The pause menu appears

☒ Success

Figure 97: Menu pause

3.2.81 Bouton Nouvelle partie

Description :

Lorsque le joueur se trouve dans le menu principal et qu'il clique sur le bouton *New Game*, il peut recommencer une partie depuis le tout début.

*Test d'acceptation :***New Game button**

Player is in the Start Menu

Player click on the *New Game* button

Player starts a New Game

☒ Success

Figure 98: Bouton nouvelle partie

3.2.82 Bouton classement des joueurs

Description :

Lorsque le joueur se trouve dans le menu principal et qu'il clique sur le bouton *Ranking*, il verra le classement des joueurs en fonction du meilleur score.

Test d'acceptation :

Ranking button

Player is in the Start Menu

Player click on the *Ranking* button

He sees a leadboard with the player and with his score

 Success

Figure 99: Bouton classement des joueurs

3.2.83 Bouton quitter

Description :

Lorsque le joueur se trouve dans le menu principal ou dans le menu pause il peut à tout moment quitter l'application en cliquant sur le bouton *Quit*.

Test d'acceptation :

Quit button

Player is in the Start Menu or the Pause Menu

Player click on the *Quit* button

The game shut down

 Success

Figure 100: Bouton quitter

3.2.84 Bouton reprendre

Description :

Lorsque le joueur est dans le menu pause, il peut reprendre sa partie actuelle en cliquant sur le bouton *Resume*.

Test d'acceptation :
Resume button

Player is in the Pause Menu

Player click on the Resume Menu

Player plays again

☒ Success

Figure 101: Bouton reprendre

3.2.85 Bouton contrôles

Description :

Lorsque le joueur est dans le menu pause, il peut consulter les contrôles du jeu en appuyant sur le bouton *Contrôles*.

Test d'acceptation :
Controls button

Player is in the Pause Menu

Player click on the *Controls* button

Player can the see the controls

☒ Success

Figure 102: Bouton contrôles

3.2.86 Bouton sauvegarder

Description :

Lorsque le joueur est dans le menu pause il peut sauvegarder manuellement sa partie.

Test d'acceptation :
Saving button

Player is in the Pause Menu

Player click on the *Saving* button

Player saved his game

☒ Success

Figure 103: Bouton sauvegarder

3.3 Codes Particuliers

3.3.1 Ennemi qui suit le Joueur :

Pour faire en sorte que l'ennemi suive le joueur peu importe sa position par rapport au joueur, je me suis inspiré d'une méthode que j'avais déjà utiliser pour un autre projet qui consiste à utiliser la librairie de math afin de calculer la distance entre les deux entités.

```
def move(self, player, collision_surface): 1 usage \(1 dynamic\) 2 riffuu1
    if not self.active:
        return # --- Do nothing if the enemy is dead ---

    dx = player.rect.centerx - self.rect.centerx
    dy = player.rect.centery - self.rect.centery

    distance = math.hypot(*coordinates: dx, dy)
    if distance == 0:
        return

    if distance <= 200: # --- Normalize vector ---
        dx /= distance
        dy /= distance

        future_hitbox = self.hitbox.copy()
        future_hitbox.x += dx * self.speed
        future_hitbox.y += dy * self.speed

        future_x = self.rect.x + dx * self.speed
        future_y = self.rect.y + dy * self.speed

        check_x = int(future_x + 64)
        check_y = int(future_y + 100)
```

Figure 104: Code du déplacement de l'ennemi

Sur cette image vous pouvez voir une partie du code du déplacement de l'ennemi, l'autre partie non visible étant lorsqu'il attaque le joueur. Le vérifie dans un premier si l'ennemi est actif, donc vivant. Une fois ce détail vérifié il se poursuit avec le calcul de la distance entre les coordonnées de l'ennemi et celles du joueur. Nous pouvons voir que la fonction `math.hypot` requiert deux variables qui sont les coordonnées `dx` et `dy` qui sont comme indiqués au-dessus, les positions du joueur. Avec ces deux données nous pouvons très facilement expliquer ce que fait la fonction par un graphique.

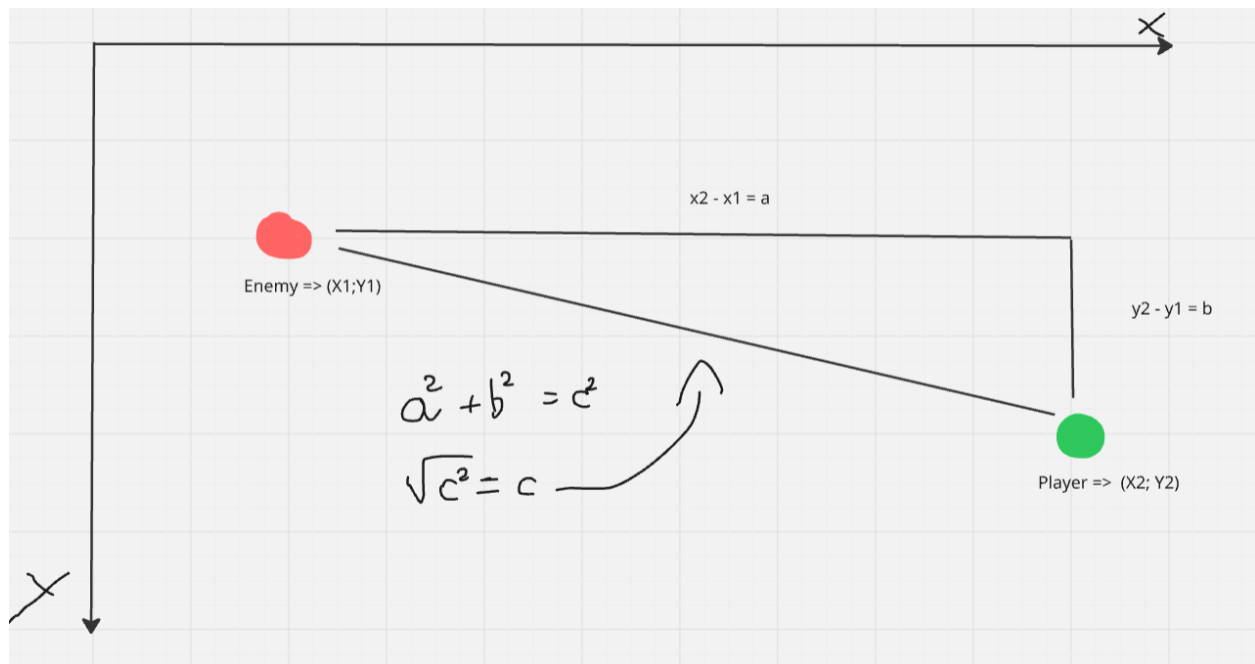


Figure 105: Schéma du calcul de la distance

Comme montré sur le graphique la fonction regarde la différence qu'il y a entre la position du joueur, ici marqué en vert, par rapport à l'ennemi, ici marqué en rouge. Comme la fenêtre est un graphique en x et y, avec son ordonnée à l'origine situé en haut à gauche de la fenêtre, il peut ainsi récupérer les coordonnées des deux entités et ainsi réaliser la formule de Pythagore pour trouver comme son nom l'indique, l'hypoténuse et ainsi calculer en tout temps la distance qui le sépare du joueur pour ensuite le poursuivre ou non.

3.3.2 Initialisation des objets dans les différentes maps :

Pour initialiser les objets dans leur map respective sans qu'ils apparaissent tous dans une même map j'ai décidé d'ajouter une liste des différents objets dans la classe Map en tant qu'attributs.

```
self.width = width
self.height = height
self.bg_image = bg_image
self.bg_name = bg_name
self.enemies = enemies if enemies else []
self.pickable_objects = pickable_objects if pickable_objects else []
self.door_objects = door_objects if door_objects else []
self.treasure_objects = treasure_objects if treasure_objects else []
self.pearls_objects = pearls_objects if pearls_objects else []
self.traps_objects = traps_objects if traps_objects else []
```

```
def draw(self, screen, camera): 8 usages (7 dynamic)  2 riffuu1
    screen.blit(self.bg_image, (-camera.x, -camera.y))

    # --- pickable objects ---
    for obj in self.pickable_objects:
        if hasattr(obj, "active") and not obj.active:
            continue
        obj.draw(screen, camera)

    # --- door objects ---
    for obj in self.door_objects:
        if hasattr(obj, "active") and not obj.active:
            continue
        obj.draw(screen, camera)

    # --- tresor objects ---
    for obj in self.treasure_objects:
        obj.draw(screen, camera)

    # --- pearl objects ---
    for obj in self.pearls_objects:
        obj.draw(screen, camera)

    for obj in self.traps_objects:
        obj.draw(screen, camera)
```

Sur ces images ci-dessus on peut voir l'initialisation des attributs « pickable, door pearl et tresor objects » sous forme de liste (la première image), et aussi comment ils sont utilisés. Nous pouvons voir que dans la fonction draw tous les objets sont dans une boucle for chacune afin de bien séparer les objets qui sont censé être dans la map spécifique celles qui ne devraient pas. En ce qui concerne la caméra elle est là pour que les objets se positionnent bien dans la map et non dans la caméra.

3.3.3 Changement de map :

Lorsque j'ai commencé à tester ma fonction « switch_map » pour changer de map, celle-ci a posé quelques difficultés du fait qu'il fallait être attentif sur la position du joueur ainsi que du background, afin de pouvoir placer la prochaine map voulu. Ces difficultés ont cependant, été résolu assez rapidement pour avoir une personne qui se déplace dans son propre monde.

```

@staticmethod 1 usage 1 riffuu1
def switch_map(current_map, player, map1, map2, map3, map4):
    # --- Border top ---
    if player.rect.y == 0:
        if current_map == map2:
            player.rect.y = 1048
            return map1
        if current_map == map3:
            player.rect.y = 1048
            return map2

    # --- Border bottom ---
    elif player.rect.y >= 1048:
        if current_map == map1:
            player.rect.y = 0
            return map2
        if current_map == map2:
            player.rect.y = 0
            return map3

    # --- Border left ---
    elif player.rect.x == 0:
        if current_map == map4:
            player.rect.x = 1468
            return map3

    # --- Border right ---
    elif player.rect.x <= 1484 and player.rect.x >= 1470 and player.rect.y >= 800:
        if current_map == map3:
            player.rect.x = 0
            player.rect.y = 950
            return map4

    return current_map

```

Figure 106: Code du changement de map

Cette image illustre la fonction « switch_map ». À l'intérieur de cette fonction on retrouve 4 blocs et chaque bloc représente un sens de direction (gauche, droite, haut, bas). À l'intérieur de ces blocs se trouve ensuite une condition if qui cherche à savoir le background actuel afin de pouvoir utiliser par la suite la prochaine bonne image.

3.3.4 Les Collisions avec hitbox :

Gérer les collisions a été un vrai défi car, au début, j'ai commencé par le faire pixel par pixel ce qui marchait assez bien pour les collisions entre le joueur et les objets mais qui a fini par poser un problème lorsque le joueur entrait en collision avec un ennemi, car à la fin il restait toujours un petit espace entre les deux entités. Pour pallier ce problème, j'ai décidé de changer de méthode de collision et ai opté pour une solution avec un système de hitbox. Le système de hitbox est utilisé dans le monde du jeu vidéo car il permet d'enfermer le personnage du joueur dans une sorte de rectangle qui gèrera les collisions dans son environnement. Dans notre cas il est

très utile car lorsque le code prend l'image, il délimite un périmètre dans lequel, il inclue non seulement ce qui a été dessiné mais aussi ce qui n'a pas été dessiné. En d'autres termes, il prend l'image dans son entièreté. En utilisant le système de hitbox, on peut modifier ce périmètre pour qu'il n'inclue que ce que l'on désire, dans notre cas simplement l'image.

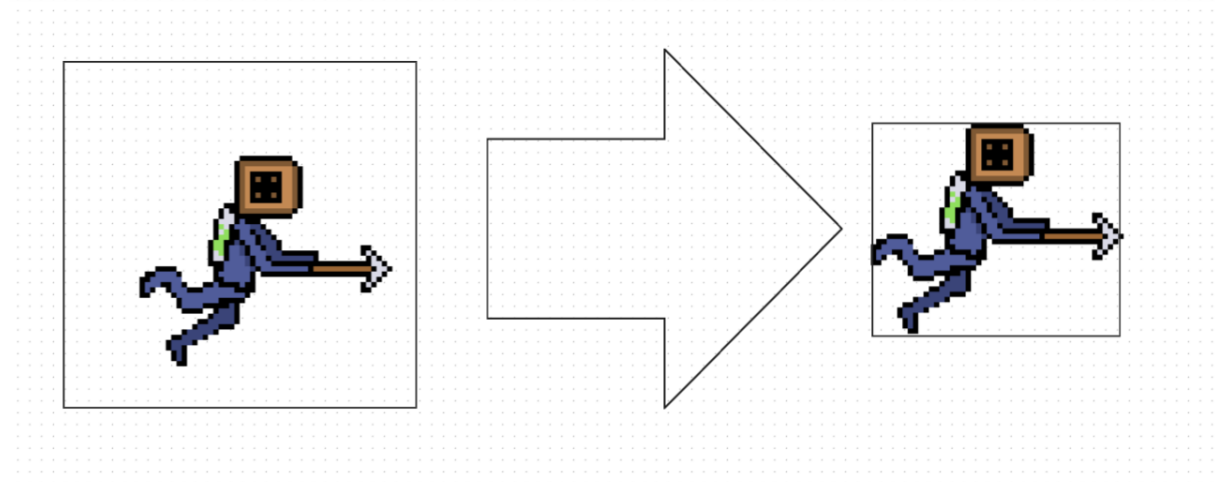


Figure 107: Représentation de la hitbox

L'image que vous voyez au-dessus illustre bien le système de hitbox. On peut voir qu'au début nous avons l'image ainsi que le reste du tableau à gauche et qu'ensuite après la mise en place du système de hitbox le rectangle rétrécit pour avoir les bonnes dimensions de l'image.

Bien sûr il reste encore un peu de vide après la réalisation de la hitbox, cependant ce vide n'est pas important car il ne se trouve pas à un endroit qui pourrait fausser la collision. Alors pourquoi ne pas continuer de le faire pixel par pixel dans ce cas ? D'une part car le temps de travail deviendrait trop long et aussi car le code deviendrait extrêmement lourd si l'ordinateur devait systématiquement calculer une collision au pixel près.

```
self.hitbox = pygame.Rect(self.rect.x + 40, self.rect.y + 70, 48, 50)
```

Figure 108: Initialisation de la hitbox

Ce code est la traduction de l'illustration au-dessus c'est ici que l'on définit clairement la hauteur ainsi que la largeur de la hitbox. Les nombres qui sont rajouté en plus du « self.rect.x » ou « self.rect.y » déterminent le point où va commencer le rectangle et les deux derniers nombres à la fin signifie la largeur (48) et la hauteur (50) de la hitbox. Ensuite pour pouvoir utiliser cet attribut, on le place dans la fonction update pour qu'il le calcule constamment lorsque le joueur se déplace.

```
old_x, old_y = self.rect.x, self.rect.y
in_movement = False

# --- Vertical moves ---
if keys[pygame.K_UP]:
    self.rect.y -= self.speed
    if self.last_direction == "left":
        self.current_animation = self.animations["walk_left"]
    else:
        self.current_animation = self.animations["walk_right"]
    in_movement = True

if keys[pygame.K_DOWN]:
    self.rect.y += self.speed
    if self.last_direction == "left":
        self.current_animation = self.animations["walk_left"]
    else:
        self.current_animation = self.animations["walk_right"]
    in_movement = True

# --- Update Hitbox ---
self.hitbox.y = self.rect.y + 70

# --- Vertical Collision ---
if self.collision(surface):
    self.rect.y = old_y
    self.hitbox.y = old_y + 70
```

Figure 109: Code exemple d'utilisation

Dans ce bout de code update on voit tout en bas l'utilisation de la hitbox. À cet endroit on décale la hitbox de sorte qu'il soit constamment à jour lorsque le joueur se déplace. Dans ce cas pour le rectangle on le déplace de 70.

3.4 Erreurs restantes

En ce qui concerne les erreurs restantes restantes voici celles qui seront traités en vue d'une prochaine release :

- *Gestion des bugs de changement de map : Dans certaines map lorsque par miracle on arrive à la limite d'une map et que celle-ci est connectée à une autre map on arriver à un glitch non bloquant ou le joueur traverse de manière infinie la map.*
- *Collision des ennemis avec les portes : Bien que les ennemis ne traverse pas les objets, ceux-ci peuvent toute fois traversé des portes lorsque le joueur se trouve à proximité de lui.*

3.5 Améliorations possibles

Plusieurs améliorations peuvent être imaginés dans plusieurs domaines. Voici certaines d'entre elle :

- Organisation
 - Workflows (vérification de la version de pygame, vérification de la syntaxe)
 - Mieux organisé prévoir plus de temps pour la documentation
- Jeu
 - Nouvelles énigmes, afin de varier les situations et rendre le jeu un peu plus stimulant
 - Nouvelles map afin de rendre le jeu plus grand avec plus d'activité à faire plus énigmes plus d'ennemis
 - Nouveaux ennemis varier les ennemis

3.6 Liste des documents fournis

Rapport TPI avec Journal de bord format PDF
Readme
Gestion de projet sur GitProject
Code du jeu sur Github

4 Conclusions

Pour conclure tous les objectifs souhaités ont été atteints j'ai pu avoir une belle idée de la quantité de travail qu'un jeu, aussi petit qu'il soit, peut représenter que ça soit du côté design ou code. Au cours de ce TPI qui s'est très bien passé j'ai pu constater que la pression imposée par les employeurs n'est pas justifiable et qu'il faudrait que le mieux que le jeu prenne le temps de sortir afin qu'il puisse être déployer sans le moindre bug plutôt que de sortir avec des bugs toujours pas résolus comme cela a été le cas avec la sortie du jeu *Cyberpunk 2077*

5 Annexes

5.1 Planification Initial illustré

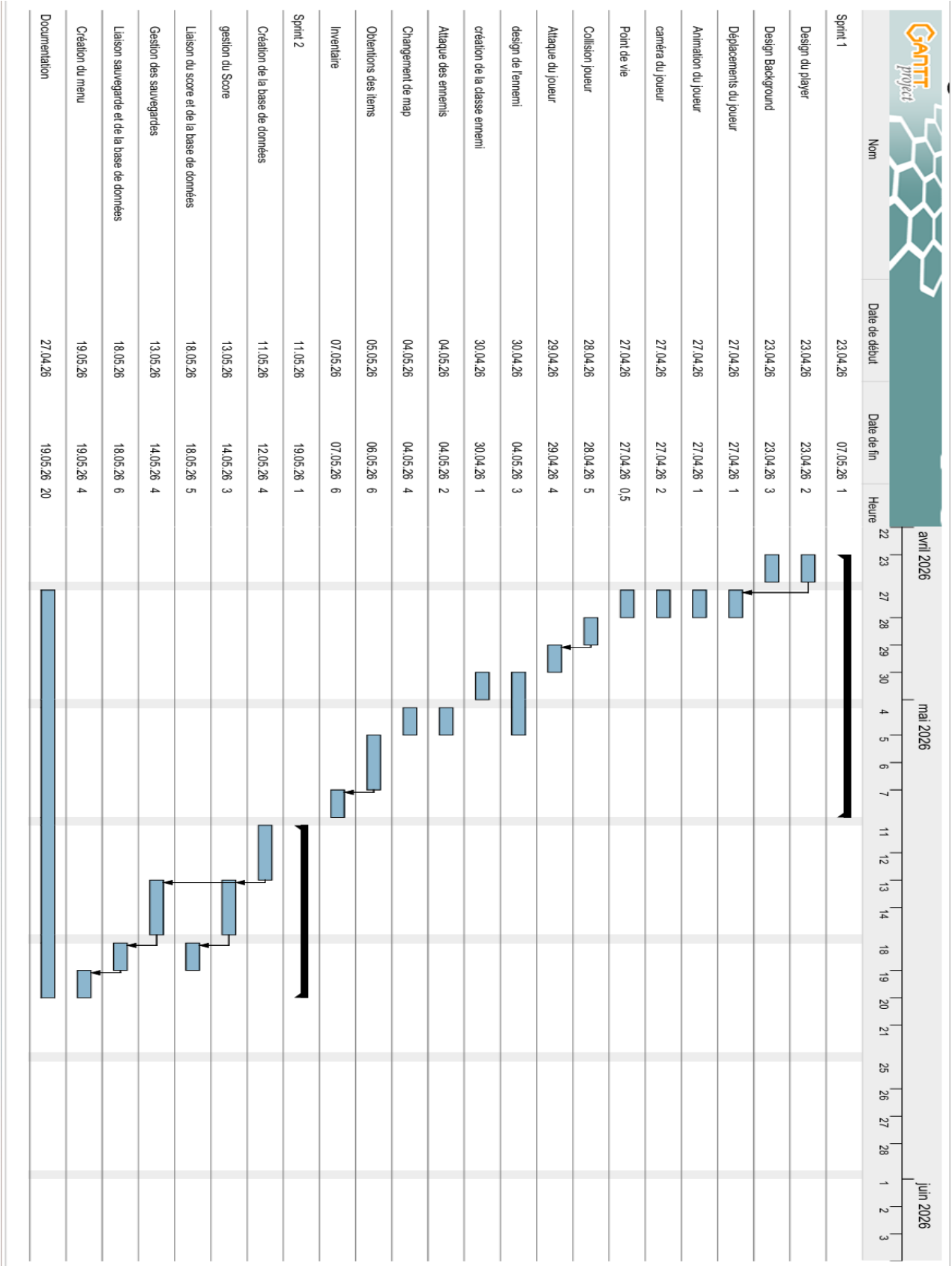


Figure 110: Diagramme de Gantt

5.2 Glossaire

Lettre	Nom	Définition
A		
	AAA	Le Triple A désigné principalement des à très gros budget, marketing et développements élevés.
B		
	Background	
C		Dans un jeu c'est l'ensemble du contexte narratif, historique et géographique d'un univers ou l'histoire passée d'un personnage, dans notre cela est un contexte géographique.
	Collision	
D		Une collision désigne la détection d'une intersection ou d'un contact physique entre deux objets virtuels.
	Diagramme de Classe	
E		C'est une représentation graphique statique issue de l'UML. Il modélise la structure du jeu en définissant les objets, leurs caractéristiques, leurs comportements et leurs relations.
	Entité	
F		C'est un élément individuel, identifiable de manière unique, qui représente un objet, un personnage, un ennemi ou un composant fonctionnel dans le monde virtuel.
	FPS	
H		FPS (First-Person Shooter) est un genre de jeu vidéo où l'action est vue à travers les yeux du personnage incarnée par le joueur. Le joueur voit ses et ses armes, mais par le reste de son corps, ce qui favorise l'immersion.
	Hitbox	C'est une zone invisible définie autour d'un personnage, d'un objet ou d'un projectile qui détermine les interactions physiques et les dégâts. C'est grâce à cette zone que le jeu calcule si un coup touche sa cible.
	Hp	C'est l'abréviation de l'anglais « Health Points » qui signifie points de vie.
I		
	Inventaire	C'est une interface ou un système (souvent un sac à dos, une mallette ou un menu) permettant au joueur de stocker, organiser, visualiser et utiliser des objets collectés, tels que des armes, consommable ou éléments de quête.
	Item	C'est un objet virtuel collectable, manipulable ou utilisable par un joueur.
M		

	Map	C'est le terrain de jeu, le niveau ou l'espace virtuel dans lequel le joueur évolue et interagit.
	Math.hypot	C'est une fonction qui calcule la racine carrée de la somme des carrés de ses arguments, représentant la norme euclidienne. En développement de jeux, elle est cruciale pour calculer rapidement des distances (2D/3D) entre entités, comme la portée d'un tir ou la proximité d'un objet.
	Menu	C'est une interface utilisateur (UI) essentielle servant de panneau de contrôle principal.
	Métroidvania	C'est un sous-genre de jeu d'action-aventure 2D caractérisé par l'exploration d'un vaste monde interconnecté et non linéaire.
P		
	Planification	C'est une fonction exécutive cognitive consistant à organiser mentalement une séquence d'actions stratégiques pour atteindre un but spécifique, comme résoudre un problème ou arriver à destination.
	Pygame	C'est une bibliothèque libre et open-source pour le langage Python, conçue pour créer des jeux vidéo 2D et des applications multimédias.
	PNJ	C'est un personnage dans un jeu vidéo ou un jeu de rôle qui n'est pas contrôlé par le joueur.
R		
	RPG	C'est un RPG est un genre de jeu vidéo où le joueur incarne un ou plusieurs personnages, les faisant évoluer dans un univers narratif.
S		
	Sauvegarde	C'est un fichier informatique enregistrant la progression d'un joueur à un instant précis.
	Sprint	C'est une itération de temps fixe (généralement 2 à 4 semaines) durant laquelle l'équipe réalise un ensemble d'incréments de produit définis à partir du Backlog.
	Sprite	C'est un élément graphique 2D, souvent animé et partiellement transparent, qui se déplace indépendamment sur le fond d'écran dans un jeu vidéo.
T		
	Test unitaire	C'est une procédure de développement logiciel qui vérifie le bon fonctionnement d'une « unité » (fonction, méthode ou classe) de code isolé, indépendamment du reste du programme.
	Test d'intégration	C'est une phase de vérification logicielle qui assemble des modules ou composants individuels pour contrôler leur bon fonctionnement conjoint.
V		
	Vecteur	C'est un objet mathématique représenté par un segment orienté, défini par une direction, un sens et une norme (longueur).

5.3 Table des illustrations

Figure 1: Image du jeu	1
Figure 2: Maquette de la page d'inscription	8
Figure 3: Maquette de la page de connexion	8
Figure 4: Maquette du jeu sans sauvegarde crée	9
Figure 5: Maquette du jeu avec sauvegarde crée	9
Figure 6: Maquette du classement des joueurs	10
Figure 7: Maquette du jeu en pleine partie	10
Figure 8: Maquette plus visuel du jeu en pleine partie	11
Figure 9: Diagramme de classes 1	12
Figure 10: Diagramme de classe 2	12
Figure 11: Diagramme de classe 3	13
Figure 12: Diagramme de classes 4	13
Figure 13: MCD	14
Figure 14: MLD	14
Figure 15: Arborescence des fichiers partie 1	19
Figure 16: Arborescence des fichiers partie 2	20
Figure 17: Arborescence des fichiers partie 3	21
Figure 18 : Déplacement vers le haut	21
Figure 19 : Déplacement vers le bas	22
Figure 20 : Déplacement vers la gauche	22
Figure 21 : Déplacement vers la droite	22
Figure 22: Affichage des points de vie	23
Figure 23: Dégâts reçus	23
Figure 24: Mort du joueur	24
Figure 25: Affichage attaque du joueur	24
Figure 26: Joueur attaque l'ennemi	24
Figure 27: Joueur tue l'ennemi	25
Figure 28: Collision du joueur avec le bord supérieur	25
Figure 29: Collision du joueur avec le bord inférieur	26
Figure 30: Collision du joueur avec le bord gauche	26
Figure 31: Collision du joueur avec le bord droit	26
Figure 32: Collision du joueur avec les objets	27
Figure 33: Déplacement de l'ennemi vers le haut	27
Figure 34: Déplacement de l'ennemi vers le bas	27
Figure 35: Déplacement de l'ennemi vers la gauche	28
Figure 36: Déplacement de l'ennemi vers la droite	28
Figure 37: Ennemi loin du joueur	29
Figure 38: Collision de l'ennemi avec le joueur	29
Figure 39: Collision de l'ennemi avec les objets	29
Figure 40: Ennemi attaque le joueur	30
Figure 41: Ennemi tue le joueur	30
Figure 42: Changement de map du haut vers le bas	31
Figure 43: Changement de map du bas vers le haut	31
Figure 44: Changement de map de la gauche vers la droite	31
Figure 45: Changement de map de la droite vers la gauche	32
Figure 46: Changement de map pas le lien vers le haut	32
Figure 47: Changement de map pas le lien vers le bas	33
Figure 48: Changement de map pas le lien vers la gauche	33
Figure 49: Changement de map pas le lien vers la droite	34

Figure 50: Récupération de l'Item et stockage dans l'inventaire	34
Figure 51: Incapacité du joueur à récupérer un item (Inventaire rempli)	35
Figure 52: Affichage de l'inventaire	35
Figure 53: Sélectionner un Item dans l'inventaire	35
Figure 54: Utiliser une potion de soin avec peu d'hp	36
Figure 55: Utiliser une potion de soin avec tous points de vie	36
Figure 56: Utiliser la clé sur la bonne porte	37
Figure 57: Inscription correcte	37
Figure 58: Inscription sans pseudo	38
Figure 59: Inscription sans mot de passe	38
Figure 60: Inscription sans confirmation de mot de passe	39
Figure 61: Possédant déjà un compte	39
Figure 62: Connexion correcte	39
Figure 63: Connexion avec mauvais pseudo	40
Figure 64: Connexion avec mauvais mot de passe	40
Figure 65: Connexion avec mauvais pseudo et mot de passe	41
Figure 66: Possédant pas de compte	41
Figure 67: Affichage du Timer	41
Figure 68: Décompte du Timer	42
Figure 69: Affichage du Score	42
Figure 70: Points gagner pour chaque ennemi battu	43
Figure 71: Augmentation du score pour chaque perle récupérée	43
Figure 72: Augmentation du score pour chaque énigme résolue	43
Figure 73: Points gagné pour chaque énigme résolue	44
Figure 74: Score bonus	44
Figure 75: Insertion du score dans la base de données lorsque le joueur meurt	45
Figure 76: Insertion du score dans la base de données lorsque le joueur gagne	45
Figure 77: Tests des deux cas où le score n'est pas mis à jour	46
Figure 78: Tests des deux cas où le score est mis à jour	46
Figure 79: Affichage des points	47
Figure 80: Augmentation de points	47
Figure 81: Affichage des pièges	48
Figure 82: Piège blessant le joueur	48
Figure 83: Piège diminuant le score	48
Figure 84: Sauvegarde automatique	49
Figure 85: Sauvegarde manuelle	49
Figure 86: ID joueur sauvegardé	50
Figure 87: Zone actuelle sauvegardé	50
Figure 88: Position du joueur sauvegardé	50
Figure 89: Points de vie du joueur sauvegardé	51
Figure 90: Inventaire du joueur sauvegardé	51
Figure 91: Score sauvegardé	52
Figure 92: Énigmes sauvegardées	52
Figure 93: Sauvegarde du temps	52
Figure 94: Charger une partie	53
Figure 95: Menu principal sans sauvegarde précédente	53
Figure 96: Menu principal avec sauvegarde précédente	54
Figure 97: Menu pause	54
Figure 98: Bouton nouvelle partie	54
Figure 99: Bouton classement des joueurs	55

Figure 100: Bouton quitter	55
Figure 101: Bouton reprendre	56
Figure 102: Bouton contrôles	56
Figure 103: Bouton sauvegarder	56
Figure 104: Code du déplacement de l'ennemi	57
Figure 105: Schéma du calcul de la distance	58
Figure 106: Code du changement de map	60
Figure 107: Représentation de la hitbox	61
Figure 108: Initialisation de la hitbox	61
Figure 109: Code exemple d'utilisation	62
Figure 110: Diagramme de Gantt	65

5.4 Résumé du rapport du TPI

Le but de mon TPI consiste à créer un jeu vidéo s'intitulant « Chroniques du Kraken oublié ». Ce jeu est un RPG en 2D de type action-explorant dans lequel nous incarnons un plongeur légendaire à la recherche d'une cité, nommé « Aquaelis », perdue au milieu de l'océan afin de mettre la main du trésor de la cité.

Dans ce jeu il y a une page de connexion incluant l'inscription pour les nouveaux afin de séparer les différents pour mieux gérer leurs sauvegardes ainsi que leur score. À la fin de la connexion le jeu retiendra l'identifiant du joueur et le mènera vers le menu principal du jeu afin de créer une nouvelle partie pour pouvoir commencer à jouer. Tout au long du jeu il y aura des obstacles à éviter des ennemis à combattre ou des énigmes à résoudre afin de pouvoir progresser dans le jeu. Les mécaniques de base tels que les déplacement le système de hitbox et le changement de map seront aussi incluses dans le jeu.

Grâce à ce TPI j'ai pu réaliser que créer un jeu vidéo, bien que celui-ci soit minuscule comparé aux géants comme le fameux triple AAA *GTA IV* ou comme le métroidevania *Hollow Knight*, prend énormément de temps et d'effort. Me mettre à la place de ces personnes qui sont constamment pressé par leurs employeurs m'a permis de comprendre que cette pression n'est pas justifiable car pour la majorité des cas le jeu s'avère être un échec et au lieu d'accuser les employeurs pour la pression qu'ils exercent, ce sont les développeurs qui se font incendier.

5.5 Sources – Bibliographie

Insérer des données dans une table MySQL avec

Python :<https://waytolearnx.com/2020/06/insérer-des-données-dans-une-table-mysql-avec-python.html> (vu le 06.05.2026)

5.6 Journal de travail

Date	Durée	Activité	Remarques
23.04.2026	45min	Rencontre avec l'expert 1 et réception du cahier des charges	
23.04.2026	15min	Planification du TPI	
23.04.2026	45min	Désign du joueur (direction et déplacement)	
23.04.2026	5 min	Meeting avec le Chef de Projet	
23.04.2026	45min	Création du GitProject avec les différentes stories ainsi que du dépôt GitHub.	
23.04.2026	15min	Planification du TPI	
23.04.2026	55min	Création des différentes stories dans GitProject	
27.04.2026	30min	Planification avec Gantt Project	
27.04.2026	45min	Documentation (Objectifs)	
27.04.2026	1h30	Déplacement du joueur avec animations	
27.04.2026	15min	GitProject (Test et réarrangement)	
27.04.2026	1h30	Maquettes	
28.04.2026	1h30	Gestion des collisions + aide de chat gpt pour les bugs	
28.04.2026	30min	Design des ennemies	
28.04.2026	15min	Class Enemy	
28.04.2026	1h30	Mouvement de l'ennemi et son attaque	
28.04.2026	1h30	Debug Mouvement et attaque de l'ennemi	
29.04.2026	45min	Debug Mouvement et attaque de l'ennemi	
29.04.2026	45min	Attaque du joueur	
29.04.2026	1h30	Design du second Backgorund	
29.04.2026	45min	Implementation du changement de map (Ajout de la classe Map)	
29.04.2026	45min	Intégration de la fonction avec le reste du jeu	
29.04.2026	45min	Documentation (Concept et Maquettes)	
29.04.2026	1h30	Documentation (Stratégie Test, Dossier de réalisation et Planification initiale)	
30.04.2026	1h30	Documentation (Introduction) +Diagramme de Classe	
30.04.2026	20min	Intégration class Objets et Items	
30.04.2026	25min	Intégration de l'inventaire	
30.04.2026	20min	Meeting avec Chef de Projet	

30.04.2026	1h10	Récolte des tests effectuées	
04.05.2026	1h30	Interaction de la Potion	
04.05.2026	2h15	Interaction porte et clé	
04.05.2026	1h30	Interaction porte et clé	Debug plus tests
04.05.2026	1h30	Documentation (Réalisation : test)	
05.05.2026	2h15	Design dernières map	
05.05.2026	1h30	Ajouts des maps et des ennemies	
05.05.2026	1h30	Documentation (Légendes plus Tests)	
06.05.2026	45min	Implémentation gestion de Score	
06.05.2026	45min	Implémentation bouton reset lors du GAME OVER	
06.05.2026	1h30	MCD + MLD	
06.05.2026	45min	Documentation (Diagramme de classes)	
06.05.2026	1h30	Documentation (Diagramme de classes, Planification, Choix du Code	
06.05.2026	45min	Gestion du GitProject	Sprints
06.05.2026	45min	Ajout de Storys GitProject	Log In /Sign In
07.05.2026	1h18	Register	
07.05.2026	32min	Visite de l'expert 2	
07.05.2026	45min	Register	
07.05.2026	25min	Meeting avec Chef de Projet	
07.05.2026	1h10	Page d'inscription	
11.05.2026	1h40	Page d'inscription	
11.05.2026	2h30	Page login avec Tests	
11.05.2026	45min	GitProject Score + fix push	
11.05.2026	2h	Time function	
11.05.2026	1h40	Implementation du trésor	
12.05.2026	1h20	Implémentation Trésor avec affiche de victoire	
12.05.2026	1h25	Intégration du score dans la base de données	
12.05.2026	3h10	Documentation : Tests	
12.05.2026	1h	Absence	Raison personnelles
13.05.2026	1h	Ajout des Pearls comme points	
13.05.2026	45min	Ajout des pièges	
13.05.2026	1h20	Gestion du Système de Sauvegarde	
13.05.2026	40min	Documentation Tests	
13.05.2026	1h	Documentation Choix du code	
13.05.2026	1h30	Organisation du code	Mise à jour du ReadMe ajout du gitignore plus debug de celui-ci
18.05.2026	1h40	Main Menu	
18.05.2026	30min	Sprint Review avec Chef de Projet	
18.05.2026	45min	Classement des joueurs	

18.05.2026	1h	Bouton Continue dans le Main Menu	
18.05.2026	50min	Fix du bouton « Don't have an account »	
18.05.2026	2h15	Documentation (Tests)	
19.05.2026	1h	Documentation (Tests)	
19.05.2026	1h40	Documentation (Conclusion, Amélioration possible)	
19.05.2026	1h40	Documentation (Résumé, Glossaire, Table des illustrations)	
19.05.2026	1h40	Documentation (Corrections avec l'aide de Chat GPT)	
20.05.2026	1h	Documentation	
20.05.2026	2h45	Implementation de plus de portes et de perles	

5.7 Manuel d'Installation

Pré-requis :

Python 3.10+

MySQL 8.0

Dépendances

Pygame : pip install pygame

MySQL- connector : pip install mysql-connector-python

Hashage du mot de passe : pip install bcrypt

Liaison entre le fichier .env et le fichier db.py : pip install python-dotenv

Lancement du programme :

- Exécuter le fichier MySQL qui se trouve dans documentation/DB/RPG2D.sql dans un client graphique pour base de données. (ex. HeidiSQL)
- Aller dans le fichier backend/config/.env.exemple et choisissez une de ces deux options
 - Copier les données collez les dans un fichier .env créé au préalable puis insérer vos identifiants afin de vous connecter à la base de données.
 - Ou alors insérer vos identifiants directement dans le fichier et renommer le fichier en .env.
- Lancer le fichier « main.py »

5.8 Manuel d'Utilisation

- Contrôles
 - Déplacer à gauche : ←
 - Déplacer à droite : →
 - Déplacer en haut : ↑
 - Déplacer en bas :
 - L'inventaire : I

- Prendre un objet : E
- Utiliser un Item : E
- Attaquer : X
- Mettre en Pause : Esc